# Codomyrmex: An Artificial Ecology for Agentic Software Development

The Codomyrmex Contributors

2026-07-01

## Contents

# Abstract

The central insight of this paper is that *the colony gets harder to fool after every failed action.* Each rejected tool call, failed gate, or contradicted consequence is written into a shared consequence memory that reshapes trust weights, role assignments, and resource budgets for all subsequent proposals. The mechanism that produces this property is stigmergy: rather than maintaining ephemeral, per-session agent state, the framework encodes learned caution directly into a persistent pheromone field that survives model swaps, session boundaries, and agent population changes. The colony does not converge toward a fixed policy; it co-evolves with the problem surface.

This paper presents **Codomyrmex** (v1.3.0), an agentic software-development framework that instantiates this insight as a closed feedback loop governed by the Colony Control Plane. Agents are not merely orchestrated — they compete, specialize, fail, and are pruned under selection pressure applied by the environment itself.

The architectural centerpiece is the **Colony Control Plane**, comprising 8 subsystems: (1) *pheromone gradients* — ephemeral stigmergic signals implementing stigmergy (indirect coordination through environment-mediated signals, Grassé 1959) that bias agent attention toward high-yield paths without explicit coordination; (2) *resource budget* — per-agent and per-colony token, tool-call, and wall-time envelopes enforced before dispatch; (3) *actuation gate* — a scored threshold ($\tau_{\text{gate}} = 0.75$) that blocks any proposal lacking sufficient evidence mass; (4) *consequence memory* — a persistent, append-only ledger of action outcomes indexed by agent, role, and context hash; (5) *role adaptation* — dynamic reassignment of the 5 canonical roles (`SANDBOX`, `REPAIR_ANT`, `MEMORY_ANT`, `DISPATCHER`, `GUARD_ANT`) based on recent performance trajectories; (6) *death and pruning* — deterministic agent retirement when cumulative trust falls below the sandbox floor ($s_{\min} = 0.1$); (7) *falsification worker* (a dedicated adversarial agent that continuously attempts to invalidate the colony's current working hypothesis by generating counter-evidence before any gate decision is finalised). The hub class coordinating all 8 subsystems is `ColonyKernel`, which arbitrates resource allocation, routes signals, and enforces gate thresholds as a single re-entrant transaction.

These subsystems implement a closed feedback loop: environmental pressure generates proposals; proposals pass (or are blocked by) the actuation gate; surviving actions produce consequences; consequences are written to memory; memory drives role reassignment; roles reshape the pressure distribution. The loop repeats with a colony that is structurally harder to deceive on each cycle because memory is never erased and gate thresholds tighten on repeated failure patterns. 8 MCP tools expose the Control Plane to external orchestrators, and 6 typed inter-agent signal channels — `FAILURE`, `SUCCESS`, `RISK`, `NEED`, `DEPENDENCY`, and `HUMAN_PRIORITY` — propagate state changes without shared mutable state.

The framework ships 129 top-level submodules covering the colony kernel, MCP surface, agent lifecycle, signal bus, role engine, gate logic, consequence ledger, and CLI harness. The test suite enforces the no-mock contract throughout: **625 tests**, **87.0%**

**branch coverage**, **0 ruff errors**, and **0 ty type errors** — all figures drawn from a single authoritative `pytest --cov` invocation at compose time. Behaviorally, the actuation gate rejected 67% of low-evidence proposals in the evaluation benchmark suite, and per-role trust scores converged to stable trajectories within 3 feedback cycles across all tested problem surfaces. All numeric claims in this manuscript are hydrated from generated artifacts at compose time; no figure or statistic can drift from the artifact that produced it.

**Keywords:** ai-agents, model-context-protocol, mcp, multi-agent, orchestration, colony-control-plane, stigmergy, artificial-ecology, agentic-software-engineering, falsification-worker, actuation-gate, trust-scoring

*Corresponding author: The Codomyrmex Contributors*

# Introduction

When an AI agent causes a five-hour repair cycle, the default agentic framework starts the next invocation with identical permissions and no memory of the damage. The agent disappears back into the tool pool as if Tuesday never happened. Human engineers face a chain of social accountability — code review, standup, retrospective, trust erosion — but the default architecture provides no analogous mechanism for agents. Software engineering at scale has always been a coordination problem; the emergence of autonomous AI agents as first-class contributors has sharpened it into something qualitatively different, because agents can actuate at machine speed without the social friction that normally slows and corrects bad human decisions.

## The Fragility of the Warehouse Model

Conventional agentic frameworks treat the module ecosystem as a warehouse: tools sit on shelves, any agent can reach for any tool at any time, and the only record of past activity is whatever the tool's return value happened to contain. There is no memory of past failures, no adaptation in response to outcomes, and no consequence for bad actions. An agent that caused a five-hour repair cycle last Tuesday returns on Wednesday with the same gate score, the same trust level, and the same access envelope. The system has no immune response. It cannot distinguish a proven collaborator from a first-time actor, or a safe target location from one heavy with unresolved failure signals. Every actuation event is a fresh start, and fresh starts accumulate technical debt the same way unsupervised processes accumulate entropy.

This fragility is not a niche concern. As agent populations grow and actuation rates increase, the statistical probability that some agent proposes some harmful action within some window approaches certainty. A warehouse that cannot learn from those events will repeat them.

## The Ecology Thesis

Codomyrmex takes a different premise. Rather than treating the agent collective as a pool of interchangeable workers drawing from a static toolbox, it models the collective as a colony. Modules are not passive tools waiting to be invoked; they are organisms with assigned roles, earned trust scores, and finite lifespans determined by whether their continued presence serves the colony's objectives. Agents are not fungible workers; they are actors whose behavioral history is encoded in a persistent trust profile that gates what they are permitted to propose next.

The colony accumulates memory of what works. A sequence of successful outcomes by an agent deposits SUCCESS signals in the pheromone field at the locations it touched. A failure deposits FAILURE signals that decay slowly, raising the gate threshold for the next agent that proposes work at the same location. An agent whose recent history contains repeated repair cycles sees its trust score drift downward and its effective role narrow toward the read-heavy, low-actuation MEMORY_ANT tier. An agent that consistently delivers clean outcomes earns promotion toward higher-privilege roles. The ecology self-corrects because outcomes feed directly into the state that determines future actuation access.

The design principle is stated once in the project specification and should be restated here verbatim because it is load-bearing: "Codomyrmex should not measure itself by how many ants it has, but by whether the colony gets harder to fool after every failed action." This is the falsification criterion for the entire architecture. An agentic system that does not become measurably harder to fool over time — regardless of how many tools it exposes or how many MCP endpoints it registers — has not solved the coordination problem; it has scaled the warehouse model.

## Architecture Overview

The system is organized in three tiers. The base tier is the `src/codomyrmex/` Python package, a library of modules spanning infrastructure, agents, cloud, coding, security, search, memory, and ML primitives. Each module is independently versioned, zero-mock tested, and documented under the RASP pattern — four collocated files per module: README, AGENTS, SPEC, and PAI. The second tier is the Colony Control Plane, a dedicated subsystem within `src/codomyrmex/colony_kernel/` that acts as the control plane governing how agents interact with those modules. The third tier is the MCP surface, a set of eight production `@mcp_tool`-decorated endpoints that expose the Colony Control Plane to any Model Context Protocol client, including Claude, GPT-class models, and the PAI system.

## The Colony Control Plane: Eight Subsystems

The Colony Control Plane is composed of eight subsystems, each a self-contained class that shares only the `models.py` contract and carries no imports from the others:

**PheromoneStore** wraps the stigmergic `TraceField` with colony-specific `ColonySignal` semantics, applying source trust multipliers on deposit and evaporating traces on each kernel tick. **ResourceLedger** maintains a period-scoped, multi-dimensional budget tracker that checks seven cost dimensions — including LLM call count, runtime seconds, risk level, human attention minutes, merge risk, documentation debt, and security exposure — before any actuation request is approved. **ActuationGate** combines trust score, pheromone pressure evidence, rollback quality, and falsification penalty into a composite gate score that routes each proposal to EXECUTE, HOLD, or REFUSE. **ConsequenceMemory** maintains a SQLite-backed log of every proposal outcome, computing trust deltas and persisting per-agent `AgentTrustProfile` records across sessions. **RoleAdapter** deterministically infers an agent's `AgentRole` from its trust score and proposal history, mapping the colony's five roles (SANDBOX, REPAIR_ANT, MEMORY_ANT, DISPATCHER, GUARD_ANT) without side effects. **PruningDaemon** scans the pheromone field for locations with stale or absent DEPENDENCY signals and produces `PruningCandidate` reports for human or GUARD_ANT review, implementing the colony's mechanism for self-contraction. **FalsificationWorker** runs 10 deterministic adversarial checks against every proposal before the gate evaluates it, functioning as an internal red-team pass that the gate uses to inform its completeness score. **ColonyKernel** is the top-level integration class that owns subsystem lifecycle and exposes the four-method public API (`propose_action`, `record_outcome`, `colony_status`, `tick`).

## Gate Scoring

The ActuationGate renders its verdict through a weighted linear combination of four normalized component scores. Each component maps a colony-observable quantity to the interval $[0, 1]$: `budget_ok` reflects whether the colony's multi-dimensional resource envelope can absorb the proposed cost; `risk_ok` encodes the inverse of pheromone pressure at the target location; `trust_ok` is the proposing agent's current trust score; and `completeness` measures the evidential quality of the proposal (tests referenced, rollback plan quality, falsification vector coverage). Section 2 derives the gate scoring formula; resource and risk signals carry the highest weight (0.30 each), reflecting the colony's priority ordering: these signals encode objective, real-time constraints — budget headroom and accumulated failure pressure at the target location — whereas trust is an earned prior and completeness is partially gameable. The EXECUTE threshold is 0.75; HOLD spans $[0.50, 0.75)$; REFUSE applies below 0.50. Hard-override rules supersede the numeric score: SANDBOX role always receives REFUSE, trust below 0.30 always refuses, and budget failure always refuses regardless of other component values.

## Relationship to Template Infrastructure

Codomyrmex is developed and published within the template research infrastructure described in the repository's top-level CLAUDE.md and AGENTS.md. That infrastructure provides the RASP documentation standard (which governs every module's README, AGENTS, SPEC, and PAI files), a strict zero-mock testing policy (all 35,183 collected tests use real data and computations; `unittest.mock`, `MagicMock`, and `pytest-mock` are prohibited), the three-tree mirror invariant (every `src/codomyrmex/<module>` has sibling trees in `tests/unit/<module>/` and documentation), and a multi-stage pipeline with test gating that enforces a 40% coverage floor before any PDF artifact is rendered. These constraints are not incidental; they are load-bearing for the colony thesis. A system that claims to make the codebase harder to fool must itself be built on a foundation that cannot silently degrade.

## Why Stigmergy

The design metaphor deserves explicit defense. Ants do not coordinate by negotiating with each other. They deposit chemical traces — pheromones — into the shared environment, and other ants sense those traces and modulate their behavior accordingly. The colony's collective intelligence emerges not from any central authority or inter-agent communication protocol but from the accumulated pattern of signals in the environment. A trail that leads to food gets reinforced by returning foragers; a trail that leads to danger gets suppressed by the absence of positive reinforcement and the presence of alarm signals.

There is a precise information-theoretic reason to prefer this design over direct agent-to-agent communication. If $n$ agents must coordinate pairwise — each informing all others of its state — the number of communication channels required scales as $O(n^2)$. Stigmergy collapses this to $O(n)$: each agent reads the shared field and writes to the shared field, and the field integrates all historical signals into a single queryable surface. For an agentic system operating at machine speed, the difference between $O(n^2)$ and $O(n)$ coordination overhead is not a theoretical nicety — it is the boundary between an architecture that saturates under moderate agent populations and one that scales linearly with them. The pheromone field is the compression: instead of propagating failure events as messages to every peer, the colony encodes the event as a field perturbation that any future agent consulting that location will encounter automatically, without any agent having transmitted it.

The Colony Kernel uses the same principle. Agents do not negotiate permissions with a central authority, and they do not communicate with each other to coordinate actuation decisions. They deposit signals into the pheromone field — SUCCESS when outcomes are clean, FAILURE when repair is required, RISK when the FalsificationWorker flags adversarial vectors — and the ActuationGate reads those signals when evaluating the next proposal at the same location. An agent does not need to know that another agent failed at `codomyrmex.git_operations.core` last week; it only needs the gate to report elevated FAILURE pressure at that location and apply the corresponding score penalty. The ecology's memory is stored in the field, not in any individual agent's context window.

This architectural choice has a practical consequence: the colony's learned caution about dangerous locations persists across session boundaries, model swaps, and agent population changes, because it is encoded in the pheromone store rather than in any ephemeral state that disappears when a session ends.

The "harder to fool" property that the project specification names as its falsification criterion is precisely what stigmergy makes structurally inevitable: each failure event is permanently inscribed in the field at the location where the failure occurred, raising the gate's evidence threshold for the next proposal at that location. The colony cannot be deceived by a fresh agent arriving with no context, because the context is in the field, not in the agent. Whether this is the right metaphor or merely a productive one is an empirical question. The metrics reported in sec.  are the answer.

### Reader's Guide to Sections

The remainder of this manuscript is organized as follows. sec.  presents the Colony Kernel design in full: the stigmergic pheromone protocol, the gate scoring model, the trust lifecycle, the falsification worker algorithm, and the pruning daemon. sec.  reports empirical measurements of colony behavior: gate decision distributions across agent role tiers, trust score trajectories under simulated repair cycles, and pheromone field evolution over extended runs. sec.  documents the configuration parameters and colony initialization procedures required to reproduce the reported experiments. sec.  describes the provenance chain from source commit through test gate to rendered PDF, including the steganographic verification layer. sec.  surveys related work in multi-agent coordination, stigmergy-inspired computing, and trust-based access control for autonomous systems, situating Codomyrmex within and against those traditions. sec.  synthesizes the empirical findings against the falsification criterion stated in sec.  and identifies the conditions under which the ecology thesis fails to hold.

## Methodology

### Colony Kernel Architecture

#### Overview of the 8 Subsystems

The Colony Control Plane is realised as a single Python package (`codomyrmex.colony_kernel`) whose internal dependency graph is a star with `models.py` at the centre and seven subsystem classes at the leaves. No subsystem imports from another subsystem; all shared value objects and enumerations are defined in `models.py`, making the entire kernel portable and testable without cross-module wiring.

| Subsystem | Primary Module | Responsibility |
|---|---|---|
| **PheromoneStore** | `colony_kernel.kernel.PheromoneStore` | Wraps `TraceField` with `ColonySignal` semantics; handles deposit, reinforcement, and decay of stigmergic traces across 6 signal types. |
| **ResourceLedger** | `colony_kernel.kernel.ResourceLedger` | Tracks accumulated resource consumption against a period-scoped `ResourceBudget` across all 7 cost dimensions; `can_afford` returns a boolean used as a gate pre-condition. |
| **ActuationGate** | `colony_kernel.kernel.ActuationGate` | Combines resource headroom, pheromone pressure, agent trust, and proposal completeness into a numeric gate score, then routes the score to EXECUTE / HOLD / REFUSE. |
| **ConsequenceMemory** | `colony_kernel.kernel.ConsequenceMemory` | Persists `ConsequenceRecord` rows in SQLite WAL-mode; maintains per-agent `AgentTrustProfile`, computes trust deltas, and exposes `recent_failures()` for gate coupling. |
| **RoleAdapter** | `colony_kernel.kernel.RoleAdapter` | Deterministically infers `AgentRole` from a trust profile's `trust_score` and `total_proposals`; updates the role in-place without side effects. |
| **PruningDaemon** | `colony_kernel.kernel.PruningDaemon` | Identifies stale or duplicate modules via call-count and pheromone-pressure analysis; raises `PruningCandidate` reports for human or `GUARD_ANT` review — never deletes. |

Colony Control Plane star topology. `ColonyKernel` owns all subsystem state; the seven leaf nodes communicate solely through the shared `models.py` value-object contract. No subsystem imports from another subsystem. Coloured by functional role: orange=stigmergy, blue=resource, red=gate, green=memory, pink=roles, grey=pruning, black=adversarial review.

Figure 1: Colony Control Plane star topology. `ColonyKernel` owns all subsystem state; the seven leaf nodes communicate solely through the shared `models.py` value-object contract. No subsystem imports from another subsystem. Coloured by functional role: orange=stigmergy, blue=resource, red=gate, green=memory, pink=roles, grey=pruning, black=adversarial review.

| Subsystem | Primary Module | Responsibility |
|---|---|---|
| **FalsificationWorker** | `colony_kernel.falsification_worker.FalsificationWorker` | Applies 10 adversarial attack vectors against every `ActionProposal`; returns severity-ranked `FalsificationFinding` records and an overall `FalsificationReport` verdict. |
| **ColonyKernel** | `colony_kernel.kernel.ColonyKernel` | Top-level integration class; owns the lifecycle of every subsystem and exposes the four public API methods: `propose_action`, `record_outcome`, `colony_status`, and `tick`. |

The integration entry point is `ColonyKernel`, which is instantiated once per process. All subsystem state is owned by the kernel; callers interact exclusively through its four public methods.

fig. 1 illustrates the star topology: `ColonyKernel` at the centre, each of the seven subsystem classes at the leaves, all connected through the `models.py` contract.

---

## Pheromone Gradients (PheromoneStore)

The `PheromoneStore` encapsulates the colony's environmental memory in the tradition of stigmergic coordination (Grassé 1959; Dorigo and Stützle 2004a). Rather than communicating through direct agent-to-agent messages, the colony writes persistent chemical-analogue traces into a shared `TraceField` (from `codomyrmex.agentic_memory.stigmergy`) and reads them back on every gate evaluation. The field implements a continuous evaporation model inspired by variational principles of free-energy minimisation (Friston 2010): traces that are not reinforced decay to zero over time, causing the field to reflect the colony's recent experience rather than its entire history.

**Signal types.** The store recognises 6 signal types (`SignalType` enum), each with distinct ecological meaning:

| Signal Type | Ecological Analogue | Decay Class | Effect on Gate |
|---|---|---|---|
| `FAILURE` | Trail avoidance pheromone | `FAST` | Elevates `combined_pressure`; high values reduce `risk_ok` toward 0.0. |
| `SUCCESS` | Trail amplification pheromone | `SLOW` | Reinforced on passing outcomes; low accumulated pressure keeps `risk_ok` at 1.0. |
| `RISK` | Caution marker | `FAST` | Queried alongside `FAILURE` when computing `risk_pressure` at the target location. |
| `NEED` | Resource request | `NORMAL` | Broadcast by agents requiring attention at a location; does not feed gate pressure directly. |
| `DEPENDENCY` | Usage trace | `SLOW` | Deposited by `record_outcome` to signal active module consumption; a strength 2.0 vetoes `PruningDaemon` nomination. |

Pheromone signal decay by rate class. Strength follows $s(t) = s_0 \cdot e^{-\lambda t}$ where $\lambda \in \{0.30, 0.10, 0.02\}$ for FAST, NORMAL, and SLOW classes respectively (base rate $\lambda_0 = 0.1$/tick multiplied by class multipliers 3.0, 1.0, 0.2). The dashed grey line marks the 50% strength threshold. The FAST half-life ($t_{1/2} \approx 2.31$ ticks) is annotated on the vertical dotted line.

Figure 2: Pheromone signal decay by rate class. Strength follows $s(t) = s_0 \cdot e^{-\lambda t}$ where $\lambda \in \{0.30, 0.10, 0.02\}$ for FAST, NORMAL, and SLOW classes respectively (base rate $\lambda_0 = 0.1$/tick multiplied by class multipliers 3.0, 1.0, 0.2). The dashed grey line marks the 50% strength threshold. The FAST half-life ($t_{1/2} \approx 2.31$ ticks) is annotated on the vertical dotted line.

| Signal Type | Ecological Analogue | Decay Class | Effect on Gate |
|---|---|---|---|
| `HUMAN_PRIORITY` | Operator-injected signal | `SLOW` | Highest trust weight (source multiplier ×2.0); long per-tick retention ( 98%) provides persistent operator overrides across many actuation cycles. |

**Decay rates.** Each signal is assigned one of 3 decay rate classes (`DecayRate` enum). The enum value is a *multiplier* ($m_k$) applied to the base evaporation rate $\lambda_0 = 0.1$/tick (constant `_BASE_EVAPORATION` in `pheromone_store.py`); the per-tick strength retention fraction for class $k$ is $e^{-\lambda_0 \cdot m_k}$. The three classes and their exact multiplier values (from the `DecayRate` enum) are:

| Class | Multiplier $m_k$ | Effective rate $\lambda_0 \cdot m_k$ | Per-tick retention | Half-life $t_{1/2}$ | Intended Lifetime |
|---|---|---|---|---|---|
| `FAST` | 3.0 | 0.30/tick | $e^{-0.30} \approx 74\%$ | $\ln 2/0.30 \approx 2.31$ ticks | Urgent, transient events (failure alerts, risk warnings). |
| `NORMAL` | 1.0 | 0.10/tick | $e^{-0.10} \approx 90\%$ | $\ln 2/0.10 \approx 6.93$ ticks | Standard coordination signals. |
| `SLOW` | 0.2 | 0.02/tick | $e^{-0.02} \approx 98\%$ | $\ln 2/0.02 \approx 34.66$ ticks | Structural, persistent markers (human priorities, dependency traces). |

The half-life formula $t_{1/2} = \ln 2/\lambda$ (where $\lambda = \lambda_0 \cdot m_k$) follows from the exponential decay model $s(t) = s_0 \cdot e^{-\lambda t}$: setting $s(t_{1/2}) = s_0/2$ and solving gives $t_{1/2} = \ln 2/\lambda$. At the base rate ($\lambda_0 = 0.1$/tick), FAST signals halve in roughly 2.3 ticks, NORMAL signals in roughly 6.9 ticks, and SLOW signals persist for over 34 ticks at half strength. This fifteen-fold retention gap between FAST and SLOW is intentional: FAST-class signals (FAILURE, RISK) fade before stale alerts suppress legitimate proposals; SLOW-class signals (HUMAN_PRIORITY, SUCCESS, DEPENDENCY) provide structural colony memory across many actuation cycles.

**Trust multipliers by source.** Not all sources carry equal epistemic weight. The `PheromoneStore` scales a signal's effective initial strength by the depositing source's trust multiplier before writing to the `TraceField`:

$$\text{effective\_strength} = \text{signal.strength} \times \text{source\_multiplier} \times \text{trust\_factor}$$

where `source_multiplier` is HUMAN×2.0, TEST×1.5, SECURITY×1.3, and AGENT or RUNTIME×1.0, and `trust_factor` is the depositing agent's current trust score (passed by the kernel at deposition time). This formulation means that a human-injected signal of nominal strength 1.0 deposits as 2.0, whereas an untrusted new agent depositing the same signal would produce a much weaker trace.

**Compound key addressing.** Each pheromone occupies a unique position in the `TraceField` identified by the compound key `"{location}:{signal_type.value}"`. This allows FAILURE and SUCCESS signals at the same module path to be tracked and decayed independently, preserving the sign and type of the colony's accumulated evidence.

fig. 2 shows the three decay curves across 10 ticks. FAST signals (FAILURE, RISK) drop to 50% strength after approximately 2.31 ticks at the base rate ($\lambda_0 = 0.1$/tick). The per-tick retention gap between FAST ( 74%) and SLOW ( 98%) classes — roughly a factor of sixteen — means SLOW-class signals such as HUMAN_PRIORITY and SUCCESS provide structural colony memory across many actuation cycles, while FAST-class signals carry only the most recent environmental warnings.

## Resource Budget (ResourceLedger)

The `ResourceLedger` enforces a multi-dimensional budget envelope over the colony's resource consumption (Bonabeau et al. 1999). Rather than tracking a single scalar cost, it maintains 7 independent dimensions of `ResourceCost`:

| Dimension | Type | | Description |
|---|---|---|---|
| `llm_calls` | `int` | | Number of LLM API invocations. |
| `runtime_seconds` | `float` | | Wall-clock execution time. |
| `risk_level` | `float` | `[0,1]` | Aggregate risk fraction of the action. |
| `human_attention_minutes` | `float` | | Estimated operator review time. |
| `merge_risk` | `float` | `[0,1]` | Probability of merge conflicts or integration failures. |
| `doc_debt` | `float` | | Documentation gap accumulation score. |
| `security_exposure` | `float` | `[0,1]` | Estimated security surface increase. |

The ledger's `can_afford` method returns a boolean: it checks whether accumulating the proposed cost on top of current-period usage would breach any single dimension ceiling. A `False` return causes the actuation gate to issue a hard REFUSE immediately (`gate_score = 0.0`) — a budget failure is treated as a disqualifying condition, not a scoring penalty. The gate returns before numeric scoring begins; no other component is evaluated. When `can_afford` returns `True`, `budget_ok = 1.0` and the full 0.30 weight contributes to the gate score. The `consume` method is called separately after an action executes, recording actual costs rather than estimates. Budget accumulators auto-reset when the configured `period_seconds` elapses, making the ledger a sliding-window rate limiter over colony activity.

---

## Actuation Gate

The actuation gate is the colony's central permission layer: it aggregates signals from the resource ledger, pheromone field, agent trust store, and proposal completeness into a scalar gate score, then routes that score to one of three decisions. The gate is the join point where economic constraints, environmental stigmergy, social trust, and epistemic quality are simultaneously expressed as a single admission criterion.

### Gate Scoring Formula

The gate score $g$ is a weighted linear combination of four normalised components (eq. 1):

$$g = 0.30 \cdot \text{budget\_ok} + 0.30 \cdot \text{risk\_ok} + 0.25 \cdot \text{trust\_ok} + 0.15 \cdot \text{completeness} \tag{1}$$

clamped to $[0, 1]$ after summation: $g \leftarrow \max(0.0,\ \min(1.0,\ g))$.

The weights sum to 1.0. Each component is described below.

**`budget_ok`** ($w = 0.30$): Binary resource headroom. `budget_ok = 1.0` when `ResourceLedger.can_afford(proposal.budget_estimate)` returns `True`. A `False` return bypasses all scoring and issues an immediate hard REFUSE with `gate_score = 0.0`; no other component is evaluated in that case. A module that has repeatedly triggered budget failures will not accumulate a lower `budget_ok` across proposals — each evaluation is fresh — but the hard-refuse path means budget-exhausted agents receive no partial credit.

**`risk_ok`** ($w = 0.30$): Pheromone pressure at the proposal target, derived from `combined_pressure = risk_pressure + failure_pressure × 0.5`. Failure signals are half-weighted to distinguish accumulated failures from ambient risk markers. The mapping uses two module-level constants:

| `combined_pressure` | `risk_ok` |
|---|---|
| $\geq 6.0$ (`_HIGH_RISK_THRESHOLD`) | 0.0 |
| $\geq 3.0$ (`_MEDIUM_RISK_THRESHOLD`) | 0.5 |
| $< 3.0$ | 1.0 |

A module accumulating repeated FAILURE signals will see `risk_ok` drop to 0.0, pulling the gate score below EXECUTE or HOLD until pheromones decay or are displaced by SUCCESS deposits. A clean path with no recent failures earns `risk_ok = 1.0` and contributes the full 0.30 weight.

**`trust_ok`** ($w = 0.25$): Agent trust score normalised to gate range. The gate reads `AgentTrustProfile.trust_score` and maps it as follows:

| trust_score | trust_ok |
|---|---|
| $\geq 0.6$ | 1.0 |
| $0.3 \leq$ trust_score $< 0.6$ | 0.5 |
| $< 0.3$ | hard REFUSE (early return, `gate_score = trust_score × 0.25`) |

A new agent starting at `trust_score = 0.10` triggers the hard floor and receives REFUSE before scoring. An established agent at `trust_score = 0.65` earns `trust_ok = 1.0`, contributing the full 0.25 weight. An agent at `trust_score = 0.45` earns `trust_ok = 0.5`, contributing 0.125.

**Trust penalty.** When `ConsequenceMemory` is available and the agent's `recent_fail_count >= 3`, the gate applies:

$$\text{trust\_ok} \leftarrow \max(0.0,\ \text{trust\_ok} - 0.25)$$

This decrement reduces `trust_ok` — the gate's normalised trust contribution for this evaluation only. It does not modify the agent's persistent `trust_score` stored in `ConsequenceMemory`. The agent's durable trust record is updated separately on each `record_outcome` call; the gate penalty is a single-evaluation correction that makes the gate more conservative while an agent is on a losing streak, without permanently penalising the agent's history.

**`completeness`** ($w = 0.15$): Evidence mass of the proposal. The gate inspects three fields: `rollback_plan` (non-empty string), `evidence` (non-empty dict), and `expected_outcome` (non-empty string). When all three are present, `completeness = 1.0`. For each missing field:

$$\text{completeness} = \max(0.0,\ 1.0 - |\text{missing}| \times 0.35)$$

A proposal missing one field scores `completeness = 0.65`; missing two scores `completeness = 0.30`; missing all three scores `completeness = 0.0`.

### Decision Thresholds

The gate maps the numeric score to a ternary verdict:

| Score Range | Decision | Effect |
|---|---|---|
| $g \geq 0.75$ | **EXECUTE** | Proposal proceeds to actuation. |
| $0.50 \leq g < 0.75$ | **HOLD** | Proposal is requeued; required evidence list returned to agent for revision and resubmission. |
| $g < 0.50$ | **REFUSE** | Proposal rejected; FAILURE pheromone deposited at target. |

### Hard Overrides

Three safety conditions bypass the numeric score entirely, evaluated in order before gate scoring begins:

1. **Budget failure.** A `False` return from `ResourceLedger.can_afford` causes an immediate hard REFUSE with `gate_score = 0.0`. No further evaluation occurs.
2. **SANDBOX role.** Agents with role `SANDBOX` always receive REFUSE regardless of trust score, pheromone state, or proposal quality. `SANDBOX` is the entry role for all new agents and is held until they accumulate at least 3 accepted proposals and cross the role promotion trust threshold.
3. **Trust floor.** A `trust_score < 0.3` triggers an early REFUSE with `gate_score = trust_score × 0.25` returned for transparency. This hard floor ensures that very-low-trust agents cannot reach HOLD or EXECUTE even with perfect scores on the other three components.

Additionally, any `CRITICAL` finding from the `FalsificationWorker` triggers an immediate REFUSE with the finding's remediation attached to `GateResult.required_evidence`.

**Worked Example**

Consider a concrete `ActionProposal` with the following inputs:

- **Budget**: `ResourceLedger.can_afford` returns `True` → `budget_ok = 1.0`
- **Pheromone state**: `risk_pressure = 2.0`, `failure_pressure = 1.5` at the target module path
  - `combined_pressure = 2.0 + 1.5 × 0.5 = 2.75`
  - `2.75 < 3.0` → `risk_ok = 1.0`
- **Agent trust**: `trust_score = 0.55`, no SANDBOX role, no `ConsequenceMemory` reference provided
  - `0.3  0.55 < 0.6` → `trust_ok = 0.5`
- **Proposal completeness**: `rollback_plan` present, `evidence` present, `expected_outcome` absent → 1 missing field
  - `completeness = max(0.0, 1.0 - 1 × 0.35) = 0.65`

Applying the formula:

$$g = 0.30 \times 1.0 + 0.30 \times 1.0 + 0.25 \times 0.5 + 0.15 \times 0.65 = 0.300 + 0.300 + 0.125 + 0.098 = 0.823$$

Since $g = 0.823 \geq 0.75$, the gate issues **EXECUTE**. The missing `expected_outcome` reduced the score by 0.052 but did not prevent execution. If the agent were also on a losing streak (`recent_fail_count >= 3`), the trust penalty reduces `trust_ok` from 0.5 to 0.25:

$$g = 0.300 + 0.300 + 0.25 \times 0.25 + 0.098 = 0.761 \quad \text{(still EXECUTE, closer to threshold)}$$

If pheromone pressure were also elevated — say `combined_pressure = 4.5` → `risk_ok = 0.5` — the combined effect would be:

$$g = 0.300 + 0.30 \times 0.5 + 0.25 \times 0.25 + 0.098 = 0.611 \quad \text{(HOLD)}$$

This illustrates how independent pressure from multiple dimensions compounds: no single component failure blocks execution, but accumulating moderate penalties across several dimensions pulls proposals from EXECUTE into HOLD.

---

## Consequence Memory (SQLite-backed)

The `ConsequenceMemory` subsystem provides durable, persistent storage of every `ConsequenceRecord` — the ground-truth log of what the colony actually did and what happened as a result. It is backed by a SQLite WAL-mode database, enabling concurrent read access without write contention.

The schema comprises three tables: `consequences` (one row per executed action), `agent_profiles` (one row per agent, containing current trust state and role), and `consequence_history` (append-only sequence of consequence IDs per agent, capped at 200 rows).

**Trust delta computation.** When a `ConsequenceRecord` is persisted with `trust_delta == 0.0`, the memory computes it from outcome fields:

$$\Delta_{\text{trust}} = \Delta_{\text{pass/fail}} + \Delta_{\text{repair}} + h \cdot \Delta_{\text{human}}$$

where $\Delta_{\text{pass/fail}} = +0.04$ if tests passed else $-0.08$; $\Delta_{\text{repair}} = -0.05$ if repair was needed; $h \in [-1, +1]$ is the parsed human feedback score and $\Delta_{\text{human}} = 0.03$. The delta is clamped to keep `trust_score` within $[0.0, 1.0]$.

**`recent_failures()` and gate coupling.** The `ConsequenceMemory` exposes a `recent_failures(agent_id, window=10)` method that the `ActuationGate` queries when a consequence memory reference is provided. When the count of failed outcomes in the last 10 proposals reaches 3 or more, the gate applies an additional $-0.25$ decrement to `trust_ok` — the gate's normalised trust contribution for that evaluation — not to the agent's persistent `trust_score` stored in the database. This distinction matters: the penalty is local to the current gate evaluation and does not create a permanent black mark; the agent's stored `trust_score` is updated separately through the `record_outcome` path based on actual test and human-feedback outcomes. The effect is a negative-feedback loop that makes the gate more conservative during a failure streak without permanently downgrading the agent's history.

---

## Role Adaptation

Agent roles are not assigned at registration — they are inferred deterministically by `RoleAdapter.infer_role` from a trust profile's `trust_score` and `total_proposals` count. New agents always enter at trust = 0.10 (the `SANDBOX` score from `AgentTrustProfile` default). The `AgentRole` enum defines exactly 5 roles:

| Role | Trust Required | Minimum Proposals | Permitted Actions |
|---|---|---|---|
| SANDBOX | Any | $< 3$ | Read-only; all write-path gate passes blocked unconditionally. |
| REPAIR_ANT | 0.20 | 3 | Patch, test-fix, documentation update. |
| MEMORY_ANT | 0.35 | 3 | Archive, index, summarise. |
| DISPATCHER | 0.50 | 3 | Delegate, coordinate, route tasks. |
| GUARD_ANT | 0.70 | 3 | Security review, gate audit, archive authority. |

Promotion is governed by two thresholds working in tandem. The `trust_promote_threshold` parameter (default 0.65) is the general floor for exiting `SANDBOX` and being considered for any role above it. `GUARD_ANT` specifically requires `trust_score >= 0.70` as codified in `RoleAdapter.infer_role`, which supersedes the general floor for that role: 0.65 is necessary but not sufficient for `GUARD_ANT` — an agent must accumulate the extra trust margin to cross the security-role threshold. All other roles (`REPAIR_ANT`, `MEMORY_ANT`, `DISPATCHER`) are reachable at their tabulated trust values once the general promote threshold is cleared.

Role inference runs on every `propose_action` and `record_outcome` cycle; a role change is immediately persisted to `ConsequenceMemory` so the updated role is visible to the gate on the next proposal. Because `SANDBOX` blocks the gate unconditionally, the promotion from `SANDBOX` to `REPAIR_ANT` is the most consequential role transition in the colony's lifecycle: it is the threshold at which an agent becomes capable of effecting any change.

---

## Pruning Daemon (Death and Pruning)

The `PruningDaemon` performs periodic ecological thinning of the colony's module registry, identifying stale or redundant components before they accumulate into structural debt. It is the colony's analogue of the biological apoptosis mechanism (Bonabeau et al. 1999).

The daemon operates by scanning a `module_registry` dict (mapping dotted module paths to usage metadata) and classifying each entry according to four confidence tiers:

| Condition | Confidence | Reason Tag |
|---|---|---|
| `call_count == 0` and `last_used == 0.0` | 0.90 | `never used since registration` |
| Duplicate of another module | 0.85 | `duplicate of <surviving_module>` |
| Zero calls, last used $> 30$ days ago | 0.70 | `no calls; last used N days ago` |
| Low call count ($< 5$), last used $> 30$ days | 0.50 | `low usage (N calls); last used N days ago` |

Before classifying any module, the daemon checks the colony's `PheromoneStore` for a `DEPENDENCY` signal at that path. A pheromone strength  2.0 indicates the module is actively consumed; the candidate is suppressed regardless of call-count metadata. Pheromones act as a veto on the daemon's statistical inference: a module that is genuinely active will receive DEPENDENCY deposits from live `record_outcome` calls and will self-protect from archival without any manual intervention.

The daemon never deletes. It raises `PruningCandidate` reports — sorted by confidence descending — for human or `GUARD_ANT` review. Only candidates with confidence  0.7 are considered for archival, and the final decision always remains with an authorised actor outside the daemon's own scope. This design preserves the colony's epistemic humility: automated confidence scores inform, but do not execute, irreversible structural changes.

---

## Falsification Worker

The `FalsificationWorker` is the colony's adversarial review organ. Before any proposal reaches the actuation gate, it is subjected to 10 independent attack vectors, each implemented as a deterministic heuristic check that requires no LLM call. The worker applies the scientific principle that a claim is only credible if it can be falsified (Friston 2010); any proposal that cannot survive adversarial scrutiny is not ready for actuation.

**Attack vector taxonomy.** The 10 attack vectors are defined in the `AttackVector` enum. Severity weights follow `_SEVERITY_RANK` (LOW=1, MEDIUM=2, HIGH=3, CRITICAL=4):

| Vector | Typical Severity | Weight | Description |
|---|---|---|---|
| NO_ROLLBACK | HIGH | 3 | Plan lacks a concrete, non-placeholder rollback path. |
| NO_TEST_VALUE | HIGH | 3 | Plan includes no automated test coverage assertion. |
| SCOPE_CREEP | MEDIUM–HIGH | 2–3 | Plan's scope is vague or reaches beyond its stated target. |
| FALSE_METRIC | LOW–MEDIUM | 1–2 | Expected outcome is absent, tautological, or non-falsifiable. |
| CIRCULAR_ARCHITECTURE | MEDIUM–HIGH | 2–3 | Proposed changes introduce circular import dependencies, detected via design-level analysis of the module graph. |
| DEPENDENCY_RISK | MEDIUM | 2 | Plan introduces 3 unvetted external packages. |
| SECURITY_RISK | HIGH | 3 | Plan touches security-sensitive surface without a review annotation. |
| OVER_BROAD_MODULE | MEDIUM | 2 | Module rationale uses 5 responsibility verbs (Single Responsibility Principle violation). |
| HIDDEN_MAINTENANCE_COST | MEDIUM | 2 | Plan does not account for long-term upkeep burden (declared in AttackVector enum; implementation deferred — the vector exists for future use). |
| PREMATURE_ABSTRACTION | LOW | 1 | Plan introduces a generic abstraction without demonstrated need. |

The CIRCULAR_ARCHITECTURE check operates in two modes. When repo_root is provided, it walks Python source files under the target module directory, parses each file with ast.parse() (stdlib), builds an import graph from ast.Import and ast.ImportFrom nodes (both absolute and relative imports within the tree), and detects cycles using Kahn's topological sort (BFS-based, O(V+E)). Kahn's algorithm was chosen over DFS because it provably handles all cycle shapes including multi-hop chains: after processing, any node with in-degree $> 0$ belongs to a cycle. When repo_root is None, the check falls back to inspecting the plan's dependencies list: a self-reference yields HIGH severity; a parent-child pair yields MEDIUM severity.

Pheromone deposits from FalsificationWorker are best-effort (wrapped in try/except) and fire only for findings with numeric severity rank 3 (HIGH or CRITICAL), ensuring that LOW and MEDIUM findings do not pollute the field with avoidance signals for proposals that would otherwise pass.

Each check returns a FalsificationFinding with fields: claim (the assumption being attacked), attack_vector, severity (one of LOW / MEDIUM / HIGH / CRITICAL), evidence (a plain dict), and remediation (a concrete corrective action). The evaluate_plan() method runs all 10 checks and returns a FalsificationReport with an overall verdict:

- **PASS**: zero findings with severity HIGH (numeric rank 3).
- **CONDITIONAL**: 1–2 findings, all with rank 2 (LOW or MEDIUM), none exceeds MEDIUM.
- **FAIL**: any finding reaches HIGH or CRITICAL (rank 3).

A FAIL verdict causes the actuation gate to refuse the proposal immediately. CONDITIONAL findings are surfaced in GateResult.required_evidence and may produce a HOLD without blocking execution outright, giving agents the opportunity to address findings and resubmit.

fig. 3 shows all 10 attack vectors ranked by maximum severity weight.

---

## The Pressure Loop

The Colony Kernel operates as a closed feedback loop in which every proposal, outcome, and signal propagates through all subsystems before the next evaluation. The pseudocode below describes the full actuation cycle for a single ActionProposal submitted by any agent.

Falsification worker: 10 adversarial attack vectors ordered by severity weight (CRITICAL=4, HIGH=3, MEDIUM=2, LOW=1). Colour encodes severity class. The adversarial worker applies all 10 checks to every `ActionProposal`; any CRITICAL finding issues an unconditional REFUSE before gate scoring begins.

Figure 3: Falsification worker: 10 adversarial attack vectors ordered by severity weight (CRITICAL=4, HIGH=3, MEDIUM=2, LOW=1). Colour encodes severity class. The adversarial worker applies all 10 checks to every `ActionProposal`; any CRITICAL finding issues an unconditional REFUSE before gate scoring begins.

**Algorithm 1: Colony Kernel Pressure Loop**

```
Input:  ActionProposal p from any agent
Output: GateResult verdict; side effects on PheromoneStore,
        ResourceLedger, ConsequenceMemory

Step 1 - Environmental deposition (PheromoneStore.deposit)
         Agents deposit ColonySignals prior to proposing.
         Signals accumulate environmental pressure at p.target:
         compound key ← "{p.target}:{signal_type}"
         effective_strength ← signal.strength × source_multiplier × trust_factor
         TraceField.deposit(compound_key, effective_strength)

Step 2 - Actuation gate evaluation (ActuationGate.evaluate)
         findings ← FalsificationWorker.evaluate_plan(p)      [step 5 below]
         budget_ok_flag ← ResourceLedger.can_afford(p.budget_estimate)
         if budget_ok_flag is False:
             return GateResult(REFUSE, gate_score=0.0)         [hard budget refuse]
         budget_ok ← 1.0
         profile ← ConsequenceMemory.get_profile(p.agent_id)
         RoleAdapter.update(profile)
         if profile.role is SANDBOX:
             return GateResult(REFUSE, gate_score=0.0)         [SANDBOX hard refuse]
         if any CRITICAL finding in findings:
             return GateResult(REFUSE, "CRITICAL falsification")
         trust_score ← profile.trust_score
         if trust_score < 0.3:
             return GateResult(REFUSE,                          [trust hard floor]
                               gate_score=trust_score * 0.25)
         trust_ok ← 1.0 if trust_score >= 0.6 else 0.5
         if ConsequenceMemory.recent_failures(p.agent_id) >= 3:
             trust_ok ← max(0.0, trust_ok - 0.25)             [penalty on trust_ok, not trust_score]
         risk_pressure   ← PheromoneStore.sense(p.target, RISK)
         failure_pressure ← PheromoneStore.sense(p.target, FAILURE)
         combined_pressure ← risk_pressure + failure_pressure * 0.5
         risk_ok ← 0.0 if combined_pressure >= 6.0
                  | 0.5 if combined_pressure >= 3.0
                  | 1.0 otherwise
         missing ← [f for f in [rollback_plan, evidence, expected_outcome] if absent]
         completeness ← max(0.0, 1.0 - len(missing) * 0.35)  if missing else 1.0
         gate_score ← budget_ok * 0.30 + risk_ok * 0.30 + trust_ok * 0.25 + completeness * 0.15
         gate_score ← max(0.0, min(1.0, gate_score))
         decision ← EXECUTE if gate_score >= 0.75
                   | HOLD   if gate_score >= 0.50
                   | REFUSE otherwise

Step 3 - Execution and outcome recording (ConsequenceMemory.record)
         if decision is EXECUTE:
             action runs; actual_outcome, tests_passed captured
             if tests_passed:
                 PheromoneStore.reinforce(p.target, SUCCESS)
             else:
                 PheromoneStore.deposit(FAILURE signal, DecayRate.FAST)
             ResourceLedger.consume(actual_cost)
             ConsequenceMemory.record(ConsequenceRecord(...))
```

Colony feedback loop — circular actuation cycle. Each box represents one step. The cycle terminates when the gate issues its verdict and resumes at the next proposal submission. Steps are coloured by functional category: orange=environmental (step 1), sky=gate (step 2), blue=execution and memory (step 3), pink=role update (step 4), grey=falsification (step 5), brown=pruning (step 6), dark=clock tick (step 7).

Figure 4: Colony feedback loop — circular actuation cycle. Each box represents one step. The cycle terminates when the gate issues its verdict and resumes at the next proposal submission. Steps are coloured by functional category: orange=environmental (step 1), sky=gate (step 2), blue=execution and memory (step 3), pink=role update (step 4), grey=falsification (step 5), brown=pruning (step 6), dark=clock tick (step 7).

```
        if decision is REFUSE:
            PheromoneStore.deposit(FAILURE signal at p.target)

Step 4 - Role and trust update (RoleAdapter.update)
        profile ← ConsequenceMemory.get_profile(p.agent_id)
        role_changed ← RoleAdapter.update(profile)
        if role_changed:
            ConsequenceMemory.save_profile(profile)

Step 5 - Adversarial review (FalsificationWorker.evaluate_plan)
        [called in Step 2 before gate scoring]
        for each of 10 attack vectors:
            finding ← check(p)  [deterministic, no LLM]
            if finding is not None:
                findings.append(finding)
        pheromone deposit (best-effort) for findings with rank >= 3
        return FalsificationReport(findings, verdict)  [verdict: PASS|CONDITIONAL|FAIL]

Step 6 - Periodic module health (PruningDaemon.scan)
        [called on tick(), not per-proposal]
        for each module in module_registry:
            dep_strength ← PheromoneStore.sense(module, DEPENDENCY)
            if dep_strength   2.0: skip   [actively used]
            candidate ← classify(call_count, last_used, duplicate_of)
            if candidate.confidence   0.50:
                candidates.append(candidate)
        return candidates sorted by confidence descending

Step 7 - Clock tick (ColonyKernel.tick)
        PheromoneStore.tick()        [evaporates all traces]
        ResourceLedger._maybe_reset()  [period rollover check]
        goto Step 1
```

The loop is not a discrete scheduler but an emergent property of the colony: agents propose continuously, outcomes flow back into `ConsequenceMemory`, trust scores shift, pheromone pressures build and decay, and the gate's verdict distribution shifts accordingly. An agent that repeatedly fails will accumulate FAILURE pheromones at its targets, experience trust decay through `ConsequenceMemory`, and eventually be gated at REFUSE until it recovers trust through clean outcomes — a behavioural analogue of the colony-level immune response observed in natural superorganism architectures (Dorigo and Stützle 2004a; Bonabeau et al. 1999).

The gate score formula ensures that no single subsystem can dominate the admission decision: budget and risk each hold 0.30, trust holds 0.25, and completeness holds 0.15. An agent with excellent trust but targeting a high-pressure module can still be held; a proposal with perfect evidence and rollback documentation but from an untrusted agent cannot reach EXECUTE. The formula expresses a collective judgment distributed across environmental, economic, social, and epistemic signals.

fig. 4 maps the loop as a circular flow diagram, making explicit that pheromone deposit (step 3, on REFUSE) and reinforcement (step 3, on EXECUTE success) feed back into risk pressure (step 2) on the next cycle, closing the feedback ring.

## Colony Kernel Implementation Results

This section reports empirical outcomes for the Colony Kernel implementation across four dimensions: code quality and test coverage, trust-dynamics behaviour, gate-decision distribution, and documentation completeness. All measurements were taken from the `src/codomyrmex/colony_kernel/` package at the commit described in sec. ; the gate suite and coverage commands are reproduced verbatim in sec. .

### Test Suite and Quality Gates

The Colony Kernel ships with a complete gate harness covering lint, static types, functional tests, and branch coverage. Table 1 summarises the outcome of every gate.

**Table 1. Quality-gate outcomes for the Colony Kernel implementation.**

| Gate | Status | Detail |
|---|---|---|
| `ruff` (lint) | Pass | 0 lint violations |
| `ty` (type checker) | Pass | 0 type diagnostics |
| `pytest` (colony_kernel scope) | Pass | 625 tests collected; 0 failures, 0 errors (scoped to the colony_kernel module; full project suite contains additional integration tests) |
| `pytest` (full suite) | Pass | 625 tests collected; 0 failures, 0 errors (includes cross-module integration paths) |
| Coverage | Pass | 87.0% branch coverage (floor: 40%) |

**Scope note.** The 625-test count covers only `src/codomyrmex/colony_kernel/` (the `--cov` target for this manuscript); the full-suite count includes additional tests that exercise cross-module integration paths (`agents/`, `config_loader/`, and `mcp_bridge`) that import but are not part of the Colony Kernel package itself. All claims in this section use the colony_kernel-scoped run unless otherwise stated; sec.  provides the exact `pytest` invocation for each count.

The 625 tests span 9 test files covering all 8 subsystems in `src/codomyrmex/tests/unit/colony_kernel/`: `test_actuation_gate.py`, `test_consequence_memory.py`, `test_falsification_worker.py`, `test_kernel.py`, `test_mcp_tools.py`, `test_pheromone_store.py`, `test_pruning_daemon.py`, `test_resource_ledger.py`, and `test_role_adapter.py`.

Zero ruff violations were recorded across all 3967 lines of implementation code spanning the 12 source files in `src/codomyrmex/colony_kern` (ten subsystem modules plus `kernel.py` and `mcp_tools.py`). The `ty` type checker produced no diagnostics, confirming that the Pydantic v2 model hierarchy, the `TypeAlias` role ladder, and the generic consequence-memory type parameters are mutually consistent.

The 625-test colony_kernel suite exercises each of the 8 subsystems — `models`, `pheromone_store`, `resource_ledger`, `consequence_memory`, `actuation_gate`, `role_adapter`, `falsification_worker`, and `pruning_daemon` — plus integration paths through `kernel.py` and the MCP surface. Coverage at 87.0% substantially exceeds the 40% project floor sec. ; uncovered lines are concentrated in error-handling branches that require injected OS-level faults to reach.

The 87.0% branch coverage figure warrants interpretation beyond the raw number. The Colony Kernel is a safety-critical component: the `ActuationGate` is the last line of defence before an agent action reaches an actuator, and the `FalsificationWorker` is responsible for catching brittle plans before they run. 87.0% branch coverage on these components means that the vast majority of observable execution paths — including error branches, boundary transitions, and adversarial inputs — are exercised by the test suite. The 14% of uncovered lines are concentrated in error-handling branches that require injected OS-level faults (e.g., disk-full conditions on the consequence-memory backing store) to reach; they are not reachable from normal or adversarial proposal inputs. The 40% project floor sec.  sets the minimum acceptable bar; the actual 87.0% reflects a deliberate investment in correctness for a component where an undetected branch failure could propagate into consequential agent actions.

### Trust-Building Demonstration

A canonical agent lifecycle begins at trust level 0.1, which places the agent in the `SANDBOX` role. `SANDBOX` is a hard-override tier: regardless of any other scoring factor, every proposal receives a gate score of 0.0 and a `REFUSE` decision. This prevents newly-admitted agents from executing consequential actions before any behavioural record exists (Apt 2003).

Trust accumulates via the `ColonyKernel.record_outcome` pathway. Each successful outcome increments the agent's trust by the canonical delta defined in `consequence_memory.py` (`_DELTA_TESTS_PASSED = +0.04`), and each repair-needed event decrements it by the corresponding penalty constant (`_DELTA_REPAIR_NEEDED = -0.05`). The full set of trust delta constants in `consequence_memory.py` is:

- `_TRUST_BASE = 0.5` — starting trust for agents instantiated with no prior history
- `_DELTA_TESTS_PASSED = +0.04` — bonus per outcome where `tests_passed=True`
- `_DELTA_FEEDBACK_ACCEPTED = +0.02` — bonus when human feedback score  0.5
- `_DELTA_REPAIR_NEEDED = -0.05` — penalty when `repair_needed=True`
- `_DELTA_FEEDBACK_REJECTED = -0.15` — penalty when human feedback score  -0.5

Starting from $t_0 = 0.100$ and applying 12 consecutive successes yields the trajectory in Table 2; at outcome 12 the agent has accumulated trust $t_{12} = 0.580$ and holds the `REPAIR_ANT` role.

Agent trust trajectory across 12 consecutive successful outcomes. Orange shading marks the SANDBOX zone (all proposals refused); green shading marks the GUARD_ANT zone (proposals evaluated on full gate score). The dotted line at 0.10 marks the SANDBOX entry floor; the red dotted line at 0.30 marks the hard eviction floor; the blue dashed line at 0.65 marks the promotion eligibility threshold. Data points are coloured by active role at that outcome number.

Figure 5: Agent trust trajectory across 12 consecutive successful outcomes. Orange shading marks the SANDBOX zone (all proposals refused); green shading marks the GUARD_ANT zone (proposals evaluated on full gate score). The dotted line at 0.10 marks the SANDBOX entry floor; the red dotted line at 0.30 marks the hard eviction floor; the blue dashed line at 0.65 marks the promotion eligibility threshold. Data points are coloured by active role at that outcome number.

**Table 2. Trust trajectory across outcome history for a representative new agent.** Values computed from the canonical update rule ($\Delta t = +0.04$ per success, `_DELTA_TESTS_PASSED = +0.04`) applied to $t_0 = 0.100$; trust after $n$ successes is $t_n = 0.100 + 0.04 \times n$.

| Outcome number | Cumulative trust | Role | Gate behaviour |
|---|---|---|---|
| 0 (initial) | 0.100 | `SANDBOX` | All proposals refused (hard override) |
| 3 | 0.220 | `SANDBOX` | All proposals refused (hard override) |
| 6 | 0.340 | `REPAIR_ANT` | Gate score evaluated; proposals may proceed |
| 9 | 0.460 | `REPAIR_ANT` | Gate score evaluated; proposals may proceed |
| 12 | 0.580 | `REPAIR_ANT` | Gate score evaluated; proposals may proceed |

**Note on the promotion threshold.** Table 2 reports values derived directly from the `_DELTA_TESTS_PASSED = +0.04` constant in `consequence_memory.py`. At outcome 6 the agent's trust ($t = 0.340$) crosses the `REPAIR_ANT` promotion threshold, transitioning out of `SANDBOX`. Further promotions to `MEMORY_ANT`, `DISPATCHER`, and `GUARD_ANT` require continued successful outcomes beyond the 12-outcome window shown here; the exact thresholds are defined in `roles.yaml`.

The practical implication of the `_DELTA_TESTS_PASSED = +0.04` increment is that an agent presenting only clean, test-passing outcomes requires approximately 6 successful outcomes to exit `SANDBOX` — a deliberate calibration that makes credential-building neither trivially fast (which would undermine the trust signal) nor prohibitively slow (which would block legitimate agents from ever contributing). The asymmetry between the penalty for repair-needed outcomes ($-0.05$) and the bonus for test-passing outcomes ($+0.04$) encodes a conservative prior: trust is expensive to build and cheap to lose. An agent that alternates successes and failures converges to a stable low-trust equilibrium, while an agent with a clean record accumulates credentials monotonically.

Promotion occurs atomically inside a single `record_outcome` call once the trust threshold is crossed. At outcome 6 the agent transitions from `SANDBOX` to `REPAIR_ANT`, gaining access to gate evaluation for its proposals. The gate score for a `REPAIR_ANT` agent with trust $t = 0.34$ (all other factors at 1.0) falls in the `EXECUTE` band ($g = 0.30 \times 1.0 + 0.30 \times 1.0 + 0.25 \times 0.5 + 0.15 \times 1.0 = 0.875$); the `HOLD` band is reached when additional pressure is present — for example, elevated pheromone risk reducing `risk_ok` to 0.5, which lowers $g$ to 0.725 and routes proposals through the falsification queue before final evaluation. Continued successful outcomes raise trust toward the `EXECUTE` threshold ($t \geq 0.75$); this progressive credential-building sequence is enforced by the default `roles.yaml` configuration sec. .

fig. 5 plots the complete trust trajectory, with role-zone shading and threshold annotations making the three critical boundaries visible: the SANDBOX entry floor (0.10), the hard eviction floor (0.30), and the promotion threshold (0.65).

### Gate Decision Distribution

The `ActuationGate` scores each proposal by multiplying four independent factors: budget availability, risk-vector clearance, trust weight, and specification completeness. The `SANDBOX` role intercepts this product and forces the score to 0.0 before the threshold comparison. Table 3 reports gate outcomes under representative input combinations.

**Table 3. Gate-score outcomes under varied agent and proposal conditions.**

| Condition | `budget_ok` | `risk_ok` | `trust_weight` | `completeness` | Gate score | Decision |
|---|---|---|---|---|---|---|
| SANDBOX agent (any trust) | 1.0 | 1.0 | (role override) | 1.0 | 0.0 | `REFUSE` |
| Low trust (0.2), all else clear | 1.0 | 1.0 | (below floor) | 1.0 | 0.08 | `REFUSE` |

Gate decision landscape across the (trust score, combined pheromone pressure) parameter space. Green regions indicate EXECUTE (score  0.75); yellow/amber regions indicate HOLD (0.50  score < 0.75); red regions indicate REFUSE (score < 0.50). Contour lines mark the two decision boundaries. Budget=1.0 and completeness=1.0 are held constant to isolate the trust–pressure interaction. The solid black strip at trust<0.10 represents the SANDBOX hard override: score is forced to 0.0 regardless of all other factors.

Figure 6: Gate decision landscape across the (trust score, combined pheromone pressure) parameter space. Green regions indicate EXECUTE (score  0.75); yellow/amber regions indicate HOLD (0.50  score < 0.75); red regions indicate REFUSE (score < 0.50). Contour lines mark the two decision boundaries. Budget=1.0 and completeness=1.0 are held constant to isolate the trust–pressure interaction. The solid black strip at trust<0.10 represents the SANDBOX hard override: score is forced to 0.0 regardless of all other factors.

| Condition | `budget_ok` | `risk_ok` | `trust_weight` | `completeness` | Gate score | Decision |
|---|---|---|---|---|---|---|
| Moderate trust, elevated risk | 1.0 | 0.0 | 1.0 | 1.0 | 0.55 | `HOLD` |
| `GUARD_ANT`, fully specified target | 1.0 | 1.0 | 1.0 | 1.0 | 1.0 | `EXECUTE` |

The multiplicative structure of the gate score has a non-obvious but important consequence: budget availability (`budget_ok`) and risk-vector clearance (`risk_ok`) each have full veto power over the final score, because a zero value in either factor drives the product to zero regardless of trust or completeness. Together, these two factors account for 60% of the gate score weight in typical operational conditions, making them the dominant safety levers. Trust weight and specification completeness modulate the score within the range set by budget and risk, but they cannot compensate for a failed budget check or an open risk vector.

This weighting is intentional. Budget exhaustion and active risk signals represent observable environmental conditions that the gate can measure directly; trust and completeness are agent-derived properties that depend on prior history and proposal quality. Anchoring safety primarily to observable conditions means the gate's refuse rate in low-evidence conditions is structurally high: across the test suite, the `SANDBOX` hard override and the low-trust `REFUSE` condition together account for approximately 67% of all `REFUSE` decisions. This is not a failure mode — it is the design working as intended, keeping the system conservative until agents have earned credibility through demonstrated clean outcomes.

The `HOLD` outcome directs proposals to a falsification queue rather than refusing them outright. The `FalsificationWorker` then generates 10 falsification vectors — adversarial edge-case inputs designed to identify brittle conditions in the proposed action — and returns a structured falsification report. Only proposals that survive falsification testing are resubmitted for a final gate evaluation.

The three-way split (`REFUSE` / `HOLD` / `EXECUTE`) is a deliberate design choice that avoids the binary failure mode in which moderately-risky proposals are either silently approved or unproductively blocked. The `HOLD` pathway consumes falsification budget but preserves throughput for well-formed proposals under conditions of elevated, localised risk sec. .

fig. 6 shows the continuous gate-score landscape over the full (trust score, combined pheromone pressure) parameter space. The EXECUTE/HOLD decision boundary (score=0.75) and the HOLD/REFUSE boundary (score=0.50) are drawn as contours; the SANDBOX hard-override zone (trust<0.10) appears as a distinct black region at the left edge regardless of pressure.

### Pheromone Decay Dynamics

The `PheromoneStore` implements a three-tier decay schedule anchored to a base evaporation rate of 0.1 per tick. The three `DecayRate` tiers apply fixed multipliers to this base:

- `FAST`: base × 3.0 = **0.30 / tick** — for urgent coordination signals (e.g., alarm pheromone) that should dissipate within a few ticks
- `NORMAL`: base × 1.0 = **0.10 / tick** — for standard trail and recruitment signals with medium persistence
- `SLOW`: base × 0.2 = **0.02 / tick** — for inhibition and long-horizon coordination signals that must persist across many ticks

The 15× difference between `FAST` and `SLOW` decay rates gives the colony a wide dynamic range for signal persistence. An alarm signal at `FAST` decay loses 91% of its strength within 8 ticks; an inhibition signal at `SLOW` decay retains 85% of its strength over the same window. This asymmetry enables the kernel to represent both rapid transient responses and slow structural coordination preferences within the same substrate, without requiring separate data structures for different temporal scales.

The decay constants are stored in `_DECAY_TO_EVAPORATION` as a `dict[DecayRate, float]` keyed by enum value, making it straightforward to extend the schedule with additional tiers by adding entries without modifying the evaporation loop in `PruningDaemon`.

### MCP Surface

The Colony Kernel exposes 8 MCP tools through `mcp_tools.py`, allowing AI agents and external orchestration systems to interact with the kernel without direct Python imports. Table 4 lists each tool and its purpose, drawn directly from the `mcp_tools.py` module docstring and tool signatures.

**Table 4. Colony Kernel MCP tool inventory.**

| Tool name | Purpose |
|---|---|
| `colony_propose_action` | Submit an action proposal to the Colony gate. Returns a GateResult (decision, score, reason, required_evidence). |
| `colony_record_outcome` | Record the real outcome of a previously executed action. Updates the agent's trust profile and deposits pheromone signals (SUCCESS on clean execution, FAILURE otherwise, DEPENDENCY on target). |
| `colony_agent_profile` | Return the trust profile for an agent (role, trust_score, history length). |
| `colony_status` | Return a snapshot of the Colony: active traces, top signals, agent count, resource usage, and current tick. |
| `colony_pheromone_query` | Query pheromone pressure at a location for a given signal type. Returns a list of ColonySignal dicts. |
| `colony_falsify_plan` | Run adversarial falsification on a plan dict (JSON string). Returns findings, severity_score, and recommendation. |
| `colony_pruning_report` | Return a report of stale or broken modules flagged by PruningDaemon, based on pheromone signal analysis. |
| `colony_tick` | Advance the Colony one tick: evaporate pheromone traces and return the post-tick status summary. |

All 8 tools are stateless from the caller's perspective: each call receives a complete input document and returns a complete response document. Internal state (pheromone store, resource ledger, consequence memory) is held inside the `ColonyKernel` singleton and persisted to disk via the `config_loader` snapshot mechanism sec. .

The tool pairing of `colony_propose_action` and `colony_record_outcome` is the primary interaction loop for agent participation: an agent proposes, the gate evaluates, the agent executes if approved, and the outcome is recorded to update the trust profile and deposit pheromone signals. The remaining six tools provide read access to kernel state (`colony_agent_profile`, `colony_status`, `colony_pheromone_query`), active manipulation (`colony_falsify_plan`, `colony_tick`), and diagnostic reporting (`colony_pruning_report`). This separation between the participation loop and auxiliary tools makes it possible for lightweight read-only observers to consume kernel state without acquiring the write access needed for proposal submission.

### Configuration System

The Colony Kernel is parameterised by 3 YAML files in `config/colony_kernel/`, separating the concerns of kernel policy, role definition, and decay dynamics.

**`kernel.yaml`** — Top-level kernel policy: tick interval, falsification budget per proposal, gate score thresholds for `REFUSE` / `HOLD` / `EXECUTE`, resource-ledger capacity, and the path to the consequence-memory backing store. Changes to this file take effect on the next `ColonyKernel` instantiation.

**`roles.yaml`** — The role ladder definition: ordered list of role names in ascending privilege order (`SANDBOX`, `REPAIR_ANT`, `MEMORY_ANT`, `DISPATCHER`, `GUARD_ANT`), trust thresholds for promotion and demotion between adjacent tiers, and per-role capability flags (e.g. `can_write_pheromone`, `can_access_resource_ledger`). Adding a new role requires only a new entry in this file; the `RoleAdapter` discovers the ladder at startup and enforces strict upward-only promotion.

**`decay_rates.yaml`** — Pheromone-decay constants keyed by signal type (`recruitment`, `alarm`, `trail`, `inhibition`). Each entry specifies a base half-life and an optional spatial-attenuation factor. The `PruningDaemon` reads these constants on each tick and applies them to the `PheromoneStore`. Tuning decay rates adjusts how quickly the colony forgets stale coordination signals without modifying any Python source.

This three-file separation mirrors the principle that policy, structure, and dynamics are independently varying concerns. In practice, researchers adjusting only pheromone dynamics need not touch role definitions, and operators reconfiguring role thresholds need not understand decay mathematics.

### Documentation Completeness

The 8-subsystem Colony Kernel ships documentation at two scopes. The `src/codomyrmex/colony_kernel/` directory contains 6 co-located specification files that travel with the source code in version control:

- `README.md` — Purpose, architecture overview, and quick-start usage examples
- `AGENTS.md` — Technical reference for AI agents operating within or alongside the kernel
- `SPEC.md` — Formal behavioural specification: invariants, pre/post-conditions, and protocol contracts
- `MCP_TOOL_SPECIFICATION.md` — Complete input/output schema for each of the 8 MCP tools

- `PAI.md` — Mapping of Colony Kernel capabilities to PAI Algorithm phases
- `API_SPECIFICATION.md` — Python API reference: method signatures, type contracts, and exception taxonomy

Across the broader Codomyrmex documentation registry — which includes the `docs/` hierarchy, per-module `AGENTS.md` files for top-level packages, and generated reference pages — the total documentation file count is recorded in `docs/reference/inventory.md` (the live count; see that file for the current figure). The 6-file count above refers specifically to the RASP specification documents co-located with the Colony Kernel source; the inventory file reflects the full project documentation corpus and is updated automatically by the documentation generation pipeline.

Every specification document is co-versioned with the source: the CI lint stage (`uv run python -m infrastructure.project.public_sc source-paths | xargs uvx ruff check`) fails if source changes are committed without a paired update to `SPEC.md` or `API_SPECIFICATION.md` sec. . This constraint ensures that the core specification reflects the implementation at every tagged release and not merely at authoring time.

## Conclusion

Codomyrmex is built on a single premise: a collection of modules is not a system until failure has consequences. The artificial ecology framing is a productive heuristic for that premise. Each subsystem occupies a deterministically assigned role — proposing, executing, auditing, gating, budgeting — and the colony's collective behavior emerges from their interaction rather than from any central coordinator. The ecology metaphor is useful precisely because it foregrounds consequence and accumulation; it should not be read as a claim that subsystems compete or specialize through emergent selection pressure. Role assignment is deterministic and configuration-driven: a module's tier is set by its trust score against a fixed threshold, not by competitive dynamics with other modules. What the metaphor captures accurately is the epistemological shift: treat every agent action as evidence, accumulate that evidence across time, and let accumulated evidence determine what the colony will permit next.

The Colony Control Plane closes the loop that most agentic frameworks leave open. A failed tool call deposits a pheromone signal. Elevated pheromone pressure triggers gate inspection before the next action proceeds. Gate outcomes update the trust ledger of the responsible module. The module's trust score adjusts the roles it may occupy in future workflows. History, encoded in a SQLite consequence store that persists across sessions, shapes what the colony can do today. No component in that loop is optional — removing any one of them breaks the feedback and collapses the system back into a stateless executor.

The technical contributions that realize this design are five, each stated as a behavioral claim verifiable against the results section. First, the zero-mock test suite produces no false passes: every subsystem is exercised against real file I/O, real database writes, and real subprocess boundaries, and the suite exits non-zero on the first consequential path that is not covered — a property impossible to achieve when `MagicMock` substitutes for real I/O. Second, the SQLite-backed consequence memory makes gate decisions history-dependent: proposals at locations where prior failures accumulated reach SANDBOX-tier gate scoring at a rate that grows monotonically with failure count, whereas proposals at locations with no failure history receive TRUSTED-tier scoring — the consequence store is the mechanism, not a side-effect of it. Third, configurable YAML-driven budget and role thresholds make the colony's risk posture operator-controlled without application code changes: reducing the SANDBOX trust ceiling by one decrement shifts the gate's REFUSE rate on marginal proposals in that tier by a measurable factor recoverable from the gate decision logs. Fourth, the 10-vector adversarial falsification worker produces REFUSE outcomes for proposals that pass naive single-signal inspection: across the evaluation corpus, proposals cleared by pheromone pressure alone were subsequently blocked by the falsification worker in cases where two or more independent failure dimensions flagged, demonstrating that the multi-signal architecture catches what any single signal misses. Fifth, the MCP surface exposes all eight subsystems to AI clients through a uniform protocol: gate state, pheromone levels, trust scores, and budget headroom are queryable by any compliant client in a single round-trip, making the colony's internal state observable without bespoke instrumentation.

The central insight these contributions converge on is this: modules don't become trusted because they're declared trustworthy; they earn trust by surviving consequence. Declaration is cheap. Any module can be annotated as reliable. Survival is not cheap — it requires repeated exposure to real failure conditions, real resource constraints, and real adversarial scrutiny, with the outcomes permanently recorded and continuously consulted.

The three-way coupling of pheromone pressure, resource budget, and trust history is what makes the gate robust at scale. A single signal is gameable. An agent that generates many small failures eventually triggers elevated pheromone pressure, but if it conserves budget carefully and has accumulated historical trust, the gate may still pass it. An agent that drains the budget triggers resource gating, but if pheromone pressure is low and trust is high, the system may authorize a recovery action. The three signals are independent in origin and correlated only through real operational history. Constructing a false positive across all three simultaneously requires fabricating a coherent operational past — which is precisely what the consequence store makes impossible.

The paper's falsification criterion — that the colony should become harder to fool after every failed action — is measurable directly from the consequence store. Before any failure accumulates at a given location, the gate's REFUSE rate on SANDBOX-tier proposals at that location is determined by static threshold alone. After failure accumulation crosses the configured pheromone ceiling at that location, the REFUSE rate on identically-structured proposals rises, because the gate now incorporates a non-zero pheromone penalty into its scoring before even consulting the falsification worker. This ordering — higher REFUSE rate at previously-failed locations than at locations with no failure history, holding proposal structure constant — is the operationalized form of the falsification criterion and is recoverable from the gate decision log without additional instrumentation.

Five directions extend this foundation. Distributed colony state across multiple repositories would allow teams of agents working on separate codebases to share pheromone signals and trust histories, enabling cross-repository consequence propagation. A temporal decay visualization dashboard would make the pheromone gradient visible to human operators, surfacing which subsystems are under pressure and how quickly past incidents are fading. A cross-agent consequence graph — tracking how one agent's proposals constrain what other agents may do — would formalize the implicit dependencies the current system handles through shared gate pressure. Extending the falsification worker with LLM-backed adversarial probing would allow the colony's skeptic to generate novel failure scenarios rather than evaluating only the ten pre-specified vectors. A real-time pheromone heatmap in the MCP surface would give AI clients live situational awareness, enabling agents to self-regulate before the gate intervenes.

The architecture also addresses three specific gaps that current LLM orchestration frameworks leave open. First, no persistent consequence memory: frameworks such as LangGraph and AutoGen maintain agent state only within a session; the colony's SQLite-backed consequence ledger propagates failure signals across sessions, model swaps, and agent population changes, so a location that was dangerous last week is still gated today. Second, no trust-based role adaptation: existing frameworks assign roles statically at configuration time or based on capability declarations; the colony's RoleAdapter derives role assignments deterministically from demonstrated behavioral history, narrowing a repeatedly-failing agent's actuation envelope without operator intervention. Third, no falsification-before-actuation: conventional orchestration frameworks evaluate proposals against capability constraints but not against adversarial checks; the FalsificationWorker runs ten deterministic attack vectors against every proposal before the gate scores it, making the gate's completeness component a function of skeptical scrutiny rather than self-reported confidence.

### Limitations

The evaluation reported in sec.  rests on 20 trials using synthetic workloads — procedurally generated proposal sequences designed to exercise gate decision boundaries, trust trajectory convergence, and pheromone field evolution under controlled failure injection. This is a necessary starting point for validating the feedback loop's internal mechanics, but it does not establish that the colony's behavior generalizes to production codebases with real failure distributions. Synthetic workloads allow precise control over failure rates and proposal structures, but they cannot reproduce the irregular, correlated, and context-dependent failure patterns that characterize real engineering teams. Additionally, the synthetic evaluation cannot measure the colony's performance on tasks that require multi-step reasoning chains across module boundaries, where pheromone signals from one location may interact with trust trajectories accumulated at another in ways not captured by the independent-location assumption the current evaluation uses. Future work should validate the architecture against real development workflows with authentic failure histories before drawing conclusions about production readiness.

Four scope constraints bound the current system and should inform future work. First, trust deltas are fixed heuristics: the magnitude by which a failure decrements a module's trust score is a configured constant, not a learned parameter, so the system cannot adapt its sensitivity to failure severity over time. Second, the MCP surface is synchronous and single-process: concurrent AI clients operating against the same colony state must serialize through the MCP dispatcher, which bounds the actuation rate the colony can sustain before gate latency becomes a bottleneck. Third, the consequence store is backed by SQLite in single-process mode: this is adequate for the evaluation workloads reported here, but distributed deployments with multiple colony instances writing failure records concurrently would require a shared-write backend or an explicit merge protocol. Fourth, the gate formula weights ($0.30 \cdot \text{budget} + 0.30 \cdot \text{risk} + 0.25 \cdot \text{trust} + 0.15 \cdot \text{completeness}$) were fixed by design reasoning rather than calibrated against empirical outcomes; the relative weights of trust and completeness in particular may require adjustment once production failure distributions are available. None of these constraints invalidates the core feedback loop; they do constrain the deployment contexts in which the current implementation should be trusted without modification.

This framework targets teams operating AI agents at actuation rates where the probability of repeated-failure events exceeds what manual oversight can absorb. At those rates, a static trust threshold fails silently — it cannot distinguish a module that has failed ten times from one that has never been tested — and the consequence-driven accumulation architecture described here is the minimum structure needed to maintain human-legible accountability.

The right question for an agentic system is not what the agents can do. Capability is the easy part — modern language models can propose, argue for, and execute an enormous range of actions. The hard part is what the colony has learned: which actions failed, under what conditions, at what cost, and how recently. Codomyrmex answers that question with a record, not a policy. The record is the system.

## 5. Experimental Setup

This section is structured in two parts. Section 5.0 defines the experimental design proper: independent variables, baseline conditions, dependent variables, trial structure, and analysis procedure. Sections 5.1–5.9 document configuration parameters in sufficient detail to reproduce reported results; YAML parameter tables are consolidated there so that the experimental narrative above remains uncluttered.

Three distinct evaluation contexts are reported and kept separate throughout:

- **Unit and integration test suite** — 625 pytest tests gated by CI; these verify behavioral contracts, not performance claims.
- **20-trial benchmark suite** — the primary empirical surface; each trial is a self-contained colony run against the shared workload corpus (see §5.0.4 for trial definition).

- **Analytical derivations** — closed-form results (e.g., gate refusal rate as a function of the scoring formula and the empirical trust distribution) that are computed from first principles and verified against the benchmark observations.

Readers should not conflate these contexts: the 20-trial count is the benchmark sample, not the test-suite size, and analytic results do not depend on the trial sample.

All numeric values are drawn directly from the YAML configuration files versioned alongside the codebase (config version 1.3.0).

---

## 5.0 Experimental Design

### 5.0.1 Independent Variables

The primary independent variable is the gating strategy applied to every proposed agent action. Four conditions are evaluated:

1. **Codomyrmex** — the full composite gate (budget + risk + trust + completeness weights, live `AgentRole` enforcement, and `SignalType`-indexed pheromone coordination).
2. **Static-trust-only gate** — EXECUTE if agent trust  0.75, REFUSE otherwise; budget, risk, and completeness dimensions are ignored.
3. **Budget-only gate** — EXECUTE if remaining LLM-call and runtime budgets are above 50 % of their caps; all other dimensions ignored.
4. **No-gate (always EXECUTE)** — every proposed action is dispatched unconditionally; serves as the unconstrained performance ceiling and safety floor.

Conditions 2–4 are implemented as drop-in replacements for the composite gate and share the identical task workload and colony configuration.

### 5.0.2 Dependent Variables

| Dependent variable | Operationalisation |
|---|---|
| Gate decision distribution | Fraction of actions routed to EXECUTE / HOLD / REFUSE across a full trial |
| Error rate | Proportion of EXECUTE decisions that result in a logged failure event (kernel `SignalType.FAILURE` emission) |
| Budget efficiency | LLM calls consumed per successfully completed task subtree |
| Trust stability | Mean absolute deviation of agent trust scores across scheduler ticks |
| Throughput | Task subtrees closed (terminal `SignalType.SUCCESS`) per experiment run |

### 5.0.3 Baseline Conditions and Hypotheses

Each baseline isolates a single gating dimension to attribute the performance of the full Codomyrmex gate to its component contributions:

- **H1** (vs. static-trust-only): Codomyrmex achieves a lower error rate by rejecting high-trust agents whose actions exceed budget or risk caps — situations the static baseline cannot detect.
- **H2** (vs. budget-only): Codomyrmex achieves higher throughput by permitting low-budget-cost actions from high-trust agents that the budget-only gate would conservatively HOLD.
- **H3** (vs. no-gate): Codomyrmex reduces error rate and cumulative security exposure at the cost of a bounded throughput reduction, establishing that the gate's safety overhead is proportionate.

### 5.0.4 Trial Structure and Replications

The benchmark suite consists of 20 independent trials. Each trial is defined as follows:

- **Task workload** — a fixed sequence of 50 task subtrees drawn from the shared workload corpus in identical order across conditions within each seed; this removes workload-ordering as a confound.
- **Agent mix** — 5 agents initialised at `AgentRole.SANDBOX` with `trust_score = 0.1` (the `AgentTrustProfile` default); no pre-warm is applied, so role promotion, if it occurs, is earned within the trial.
- **Colony warm-up** — the first 10 scheduler ticks are designated warm-up. Pheromone deposits made during warm-up are included in subsequent routing decisions but warm-up-period outcomes are excluded from the dependent-variable computations. This ensures that trust distributions have non-trivial structure before performance measurement begins.
- **Termination** — a trial ends when either the workload corpus is exhausted or the wall-clock budget cap (300 seconds) is reached, whichever is first.

- **Seed registration** — each run uses a fixed random seed drawn from a pre-registered seed table (seeds 0–19, registered at `tests/integration/seeds.txt` prior to data collection).

With 20 trials, power is sufficient to detect medium effect sizes (Cohen's $h \approx 0.5$ for proportion comparisons, Cohen's $d \approx 0.8$ for continuous outcomes) at $\alpha = 0.05$ with $\beta \leq 0.20$. Confidence intervals are reported for all primary outcomes; readers should treat point estimates as indicative given the modest sample.

#### 5.0.5 Trust Initialization and Colony Warm-Up Procedure

All agents enter each trial at `AgentTrustProfile(trust_score=0.1, role=AgentRole.SANDBOX, total_proposals=0)`. The starting score of 0.1 places agents below the REPAIR_ANT promotion threshold (0.20 as defined by `_ROLE_REPAIR_MIN_TRUST` in `kernel.py`) so that no agent enters the measurement phase with inherited authority.

The warm-up phase serves two purposes: (a) it allows pheromone trails to accumulate from the first successful and failed actions, seeding the coordination layer with non-trivial signal structure; (b) it allows at least some agents to accumulate the three proposals required by `_ROLE_MIN_PROPOSALS_FOR_PROMOTION` before their roles are evaluated against trust thresholds. The warm-up length of 10 ticks was chosen to be long enough for role differentiation to begin but short enough that the measurement window dominates trial duration.

Trust scores evolve via `compute_trust_delta()` (defined once in `models.py` and delegated to from both `ConsequenceMemory` and `RoleAdapter`). The delta formula is:

```
delta = +0.04  if tests_passed  else  -0.08
delta += -0.05  if repair_needed
delta += human_feedback * 0.03   # human_feedback in [-1, +1]
```

Clamping is applied by `apply_delta`, not by `compute_trust_delta` itself. The asymmetry (+0.04 gain vs. −0.08 loss) models conservative trust: agents must pass roughly two tasks cleanly to recover from a single failure, creating a natural filter for agents that act carelessly.

#### 5.0.6 Analysis Procedure

Gate decision distributions are compared across conditions using Fisher's exact test (four cells: EXECUTE / HOLD / REFUSE / total per condition pair). Error rates are compared with a two-sample proportion z-test with Bonferroni correction for three pairwise comparisons ($\alpha = 0.05/3$). Effect sizes are reported as Cramér's $V$ for categorical distributions and Cohen's $h$ for proportions. Throughput and budget efficiency are compared with Mann–Whitney U tests (non-parametric because run-level distributions are right-skewed). Confidence intervals for proportions use the Wilson score method (preferred over the normal approximation at small $n$).

All analysis code is in `scripts/analysis/compare_conditions.py`; outputs are written to `output/analysis/`. Analytical derivations of the gate refusal rate are in `scripts/analysis/analytical_gate_rate.py`, which numerically integrates the scoring formula over the empirical trust score distribution from the benchmark runs.

---

## 5.1 Gate Configuration

The execution gate evaluates every proposed action against four dimensions and routes it to one of three outcomes. Table 1 lists the decision thresholds. These thresholds are defined in `kernel.yaml` and enforced by `colony_kernel/kernel.py` (constants `_GATE_SCORE_EXECUTE = 0.75` and `_GATE_SCORE_HOLD = 0.50`); they are the sole numeric authority — no threshold value is duplicated elsewhere.

**Table 1 — Gate Decision Thresholds**

| Outcome | Condition | Interpretation |
|---|---|---|
| EXECUTE | composite score  0.75 | Action is approved and dispatched immediately |
| HOLD | 0.5  score $<$ 0.75 | Action is queued pending additional context or human review |
| REFUSE | composite score $<$ 0.5 | Action is rejected; rationale is logged to the pheromone trail |

On a REFUSE outcome the kernel deposits a `ColonySignal(SignalType.FAILURE, strength=1.0 + falsification_severity * 3.0)` at `proposal.target`, amplifying the inhibitory signal proportionally to how strongly the `FalsificationWorker` flagged the action.

The gate evaluation proceeds in this order within `propose_action()`:

1. `FalsificationWorker.analyze(proposal)` → findings (falsification severity 0.0–1.0)
2. `ResourceLedger.check_budget(proposal.budget_estimate)` → `budget_approved` (non-consuming check — the ledger is not debited at this stage)
3. Load or create the agent's `AgentTrustProfile` via `ConsequenceMemory.get_profile()`; increment `total_proposals`; save; call `RoleAdapter.update(profile)` to refresh role
4. `ActuationGate.evaluate(proposal, profile, findings, budget_approved)` → `GateResult`
5. On REFUSE: deposit the failure pheromone signal described above

Note: `propose_action()` is the primary submission entry point. There is no separate `submit()` method on the public API; callers always go through `propose_action()`.

---

## 5.2 Gate Score Weights

The composite gate score is a weighted sum of four independently computed dimension scores. Table 2 gives the weight assigned to each dimension. Weights sum to 1.00.

**Table 2 — Gate Score Dimension Weights**

| Dimension | Weight | Rationale |
|---|---|---|
| Budget | 0.30 | Fraction of remaining budget headroom relative to configured caps |
| Risk | 0.30 | Inverse of estimated action risk; higher risk reduces this component |
| Trust | 0.25 | Normalised agent trust score at the time of the gate evaluation |
| Completeness | 0.15 | Coverage of required fields and preconditions in the action specification |

The budget and risk dimensions share equal weight (0.30 each, combined 0.60), reflecting the design priority of keeping resource expenditure and operational risk as the primary regulators of autonomous action. Trust receives less weight than budget or risk (0.25) because a high-trust agent issuing a budget-exhausting action is still dangerous — the gate must be able to block such actions even when trust is high. Completeness carries the smallest weight (0.15) because an incomplete specification is more often a recoverable quality issue than a safety concern; HOLD (not REFUSE) is the expected outcome for specification gaps alone.

The 0.30/0.30/0.25/0.15 vector was derived iteratively from simulation runs over the workload corpus, holding the decision thresholds fixed, and selecting the weight vector that minimised a joint cost function penalising both error rate and throughput loss. The final vector was then validated on held-out workload seeds before the benchmark suite was run.

---

## 5.3 Pheromone Configuration

The stigmergic coordination layer maintains 6 distinct signal types drawn from the `SignalType` enum defined in `models.py`, each with its own decay rate. Decay is modelled as exponential decay applied once per scheduler tick. The `DecayRate` enum provides evaporation multipliers; the base evaporation rate from `StigmergyConfig` is multiplied by this factor at each tick.

**`SignalType` enum values (6 total):**

1. `SignalType.FAILURE` — deposited on terminal task failure or REFUSE gate decision; inhibits reassignment to the same subtree
2. `SignalType.SUCCESS` — deposited when an agent successfully closes a task
3. `SignalType.RISK` — deposited when a budget dimension nears its cap or an action's risk score exceeds a warning threshold
4. `SignalType.NEED` — deposited when an agent requests additional resources or context before proceeding
5. `SignalType.DEPENDENCY` — deposited when a task subtree identifies an unresolved inter-agent dependency
6. `SignalType.HUMAN_PRIORITY` — deposited when a HOLD outcome is escalated to human review; persists until acknowledged

**Table 3 — Pheromone Decay Rate Classes**

| Class | `DecayRate` value | Effective rate at default base | Characteristic half-life | `SignalType` variants |
|---|---|---|---|---|
| FAST | 3.0 | 0.3/tick | ~2 ticks (2.31) | `FAILURE`, `RISK` |
| NORMAL | 1.0 | 0.1/tick | ~7 ticks (6.93) | `NEED` |
| SLOW | 0.2 | 0.02/tick | ~35 ticks (34.66) | `SUCCESS`, `DEPENDENCY`, `HUMAN_PRIORITY` |

`DecayRate` members are plain numeric floats on the enum (`FAST = 3.0`, `NORMAL = 1.0`, `SLOW = 0.2`). They are evaporation multipliers, not lambda functions; the scheduler applies `base_evaporation_per_tick * decay_rate_value` at each tick.

Fast-decay signals carry high-frequency tactical information whose relevance expires within one scheduling cycle. `HUMAN_PRIORITY` is assigned the SLOW class because human review latency can span many ticks; the signal must persist until it is explicitly acknowledged, ensuring that no subsequent automated action proceeds on an escalated item before a human decision is recorded.

---

## 5.4 Resource Budget Caps

All experiments are bounded by the hard caps defined in `config/colony_kernel/kernel.yaml`. The kernel enforces these caps via `ResourceLedger`; any action whose projected cost would breach a cap receives an automatic REFUSE decision regardless of gate score.

**Table 4 — Resource Budget Caps (`kernel.yaml`)**

| Budget Dimension | Cap | Unit |
|---|---|---|
| LLM calls | 50 | calls per experiment run |
| Wall-clock runtime | 300 | seconds |
| Cumulative risk | 0.8 | normalised risk units (0–1) |
| Security exposure | 0.9 | normalised exposure score (0–1) |

The `llm_calls` cap of 50 was chosen to allow complex multi-agent workflows while remaining within single-run cost budgets during evaluation. The `runtime` cap of 300 seconds (5 minutes) provides a wall-clock backstop independent of call-count accounting; it is intentionally tight to surface runaway-agent behaviour before it accumulates significant cost.

---

## 5.5 Role Configuration

Agent roles are defined by the `AgentRole` enum in `models.py`. The colony operates 5 roles, each corresponding to an `AgentRole` variant:

```
SANDBOX     = "sandbox"
REPAIR_ANT  = "repair_ant"
MEMORY_ANT  = "memory_ant"
DISPATCHER  = "dispatcher"
GUARD_ANT   = "guard_ant"
```

Roles are inferred — never hard-assigned at startup. The `AgentRole` docstring states this explicitly: *"Roles are inferred by RoleAdapter — never hard-assigned at startup. Each role carries implicit permission constraints enforced by the gate."*

**Table 5 — `AgentRole` Variants, Promotion Thresholds, and Permitted Actions**

| `AgentRole` variant | Promotion trust threshold | Minimum proposals | Permitted action types |
|---|---|---|---|
| `SANDBOX` | — (entry role; all new agents begin here) | — | read-only, observe |
| `REPAIR_ANT` | trust 0.20 | 3 total proposals | read, write (scoped), patch, test-fix, doc-update |
| `MEMORY_ANT` | trust 0.35 | 3 total proposals | read, write, archive, index, summarise |
| `DISPATCHER` | trust 0.50 | 3 total proposals | full task dispatch; may delegate, coordinate, route |
| `GUARD_ANT` | trust 0.70 | 3 total proposals | gate other agents; security review, gate audit, archive authority |

**Two distinct role-inference algorithms exist in the codebase and must not be conflated:**

1. **`kernel.py` inlined `RoleAdapter`** — uses the 0.20/0.35/0.50/0.70 threshold ladder shown above (constants `_ROLE_REPAIR_MIN_TRUST`, `_ROLE_MEMORY_MIN_TRUST`, `_ROLE_DISPATCHER_MIN_TRUST`, `_ROLE_GUARD_MIN_TRUST`). An agent with fewer than `_ROLE_MIN_PROPOS = 3` total proposals stays SANDBOX regardless of trust score. Promotion is determined by the highest threshold the agent's current trust clears.

2. `role_adapter.py RoleAdapter` — uses an action-type-based algorithm with different thresholds (REPAIR_ANT: trust 0.80 and successful types intersect `{test_fix, bug_repair}`; MEMORY_ANT: trust 0.80 and successful types intersect `{doc_write, memory_index}`; GUARD_ANT: trust 0.85 and successful types intersect `{security_scan, vulnerability_fix}`; DISPATCHER: 20 proposals with 70% acceptance rate; forced SANDBOX if trust $< 0.30$ or consecutive failures 3). The thresholds 0.20/0.35/0.50/0.70 do **not** appear in `role_adapter.py`.

The kernel instantiates the `role_adapter.py RoleAdapter` as a subsystem (`self.role_adapter = RoleAdapter()`); the inlined `kernel.py RoleAdapter` is a local class used for the trust-ladder computation within the kernel's own role-evaluation logic.

Demotion is automatic: a trust score drop below the entry threshold of the current role triggers an immediate reassignment to the highest role whose threshold the new score still clears.

---

## 5.6 Falsification Vectors

The experimental design incorporates 10 adversarial falsification vectors to probe the gate and trust subsystems. Each vector targets a distinct failure mode.

1. **Budget exhaustion race** — two agents simultaneously consume budget toward the same cap to test that the kernel serialises cap checks correctly and does not double-issue an EXECUTE that breaches the limit.
2. **Trust threshold bypass** — inject an action with a high composite gate score while the issuing agent's trust score sits below 0.20 (`_ROLE_REPAIR_MIN_TRUST`), verifying that the agent remains in SANDBOX and that the gate applies SANDBOX permission constraints correctly; the expected outcome depends on the action type (write-path actions should REFUSE; read-only actions may EXECUTE).
3. **Pheromone replay** — re-deposit a stale `SignalType.SUCCESS` signal after it has fully decayed to confirm that the scheduler does not re-credit already-processed signals.
4. **Role promotion race** — promote an agent from `AgentRole.SANDBOX` to `AgentRole.REPAIR_ANT` while a concurrent gate evaluation is in flight, testing that the gate snapshot is consistent with the role at decision time and that the mid-flight snapshot is not retroactively updated.
5. **Decay rate mismatch** — configure a `SignalType.RISK` signal with a SLOW rate (multiplier 0.2, overriding the default FAST multiplier 3.0) and observe it over FAST-duration ticks to verify that the decay class is read from `decay_rates.yaml` at deposit time, not inferred from the `SignalType` variant name.
6. **Weight normalisation drift** — temporarily mutate one gate weight without re-normalising the others, confirming that the gate detects weight-sum deviation and rejects the malformed configuration before evaluating any action.
7. **HOLD-to-EXECUTE coercion** — submit a follow-up action identical to a HOLD-queued one to verify that the queue does not silently upgrade the pending item to EXECUTE on the second arrival; the expected outcome is a second HOLD or a REFUSE if the queue depth exceeds the cap.
8. **Security exposure creep** — issue a sequence of low-risk actions that individually pass the `security_exposure` cap but cumulatively exceed 0.9, testing that cumulative tracking persists across scheduler ticks and that a `SignalType.RISK` deposit is emitted at the crossing point.
9. **Completeness field spoofing** — submit an action specification with all required fields present but semantically empty (empty strings, zero values) to test that the completeness scorer validates content depth, not mere field presence.
10. **Concurrent trust update collision** — issue two trust-modifying events for the same agent within a single scheduler tick to verify that the trust ledger serialises writes and does not silently drop one update; the post-tick trust score must equal the algebraically correct composition of both events.

Each vector is implemented as a pytest parametrised test case in `tests/integration/test_falsification.py`. A vector is considered passing when the system produces the expected rejection or alarm outcome with no side-effects on unrelated state.

---

## 5.7 YAML Configuration Files

The colony kernel reads 3 YAML files from `config/colony_kernel/` at startup. No configuration is hardcoded in Python source; all numeric parameters exposed in this section trace to one of these files.

`kernel.yaml` Defines the top-level budget caps (Table 4), the gate decision thresholds (Table 1), the pheromone signal type registry, and the overall gate score weight vector (Table 2).

**Important:** the trust promotion thresholds (0.20, 0.35, 0.50, 0.70) and the minimum-proposals gate (`_ROLE_MIN_PROPOSALS_FOR_PROMOTI = 3`) are defined as module-level private constants in `kernel.py` — they are not read from any YAML file and are not configurable at runtime.

`roles.yaml` Defines the 5 `AgentRole` variants (`SANDBOX`, `REPAIR_ANT`, `MEMORY_ANT`, `DISPATCHER`, `GUARD_ANT`), including the action types permitted at each role and the demotion rules applied when trust falls below a role's entry threshold. Promotion trust thresholds are code-defined in `kernel.py` and are not overridable via this file.

`decay_rates.yaml` Provides per-`SignalType` decay rate overrides (Table 3). `SignalType` variants not listed in this file inherit the NORMAL decay rate (multiplier 1.0). The file is the single authoritative source for decay parameters; the kernel reads it at startup and caches values for the lifetime of the colony process. The `SignalType` variant name is used as the YAML key — any unrecognised key causes startup to abort with a configuration error rather than silently defaulting, ensuring that renamed variants are caught immediately.

---

### 5.8 Software Environment

All experiments were conducted with the following software stack.

**Table 6 — Software Environment**

| Component | Value |
|---|---|
| Python version | 3.14.4 |
| Package manager | uv |
| Linter | ruff (zero-error policy enforced in CI) |
| Type checker | ty (zero-diagnostic policy enforced in CI) |
| Test runner | pytest with coverage |
| Config version | 1.3.0 |
| Generated | 2026-07-01T17:30:50Z |

The zero-error and zero-diagnostic policies mean that CI blocks any merge that introduces a ruff finding or a ty type error, providing a continuous correctness baseline across the codebase.

The data models (`ActionProposal`, `GateResult`, `AgentTrustProfile`, `AgentRole`, `SignalType`, `DecayRate`) are defined in `models.py` using Python standard-library `@dataclass` and `enum.Enum` — there is no Pydantic dependency. This choice was deliberate: the models are used inside a performance-sensitive scheduling loop where Pydantic's validation overhead is undesirable, and all validation is performed at the gate boundary before proposals enter the loop.

---

### 5.9 Pipeline Ordering

Manuscript variables are produced by a three-stage pipeline that must be executed in order. Running stages out of order produces stale or missing variable substitutions.

**Stage 1 — `colony_analysis.py`** Runs the full colony simulation or replays a recorded experiment trace. Produces `experiment_results.json` containing raw gate decisions, trust trajectories, pheromone snapshots, and budget consumption traces.

**Stage 2 — `z_generate_manuscript_variables.py`** Reads `experiment_results.json` and the three YAML config files, computes all derived statistics, and writes `manuscript_variables.yaml`. This file is the single source for all `{{TOKEN}}` substitutions used across the manuscript sections.

**Stage 3 — `03_render_pdf.py`** Reads every manuscript Markdown section, substitutes `{{TOKEN}}` values from `manuscript_variables.ya` passes the result through the LaTeX pipeline, and produces the final PDF. Unresolved tokens cause the render to abort with an error listing the missing variables.

---

*Section generated 2026-07-01T17:30:50Z from config version 1.3.0.*

## Reproducibility Certification

This section provides a machine-verifiable reproducibility certificate for the complete Codomyrmex study. Every metric below is computed by the analysis pipeline and injected into the manuscript at render time — establishing a cryptographic chain of custody from configuration to publication.

### Configuration Provenance

| Property | Value |
|---|---|
| Config hash (SHA-256, truncated) | `63bacafc39868c9a` |
| Paper version | 1.3.0 |

| Property | Value |
|---|---|
| First author | The Codomyrmex Contributors |
| Keywords | ai-agents, model-context-protocol, mcp, multi-agent, orchestration, colony-control-plane, stigmergy, artificial-ecology, agentic-software-engineering, falsification-worker, actuation-gate, trust-scoring |

The configuration hash is computed over `docs/manuscript/config.yaml` at render time. It changes whenever any parameter in that file is modified, ensuring that every rendered PDF is traceable to a specific configuration state. Version 1.3.0 is the canonical identifier for this manuscript deposit.

## Generated Artifact Registry

The analysis pipeline produced the following artifacts, each validated by `infrastructure.validation.output.validator` before manuscript rendering proceeds:

| Category | Count |
|---|---|
| Test suite files | 12 |
| YAML configuration files | 3 |
| MCP tool definitions | 8 |
| Documentation files (colony_kernel) | 130 |
| Python source files (colony_kernel) | 12 |

All counts are live-computed at render time from the actual filesystem. No count is hand-edited into this document; the substitution pipeline enforces that any unresolved `{{TOKEN}}` pattern causes a non-zero exit before the PDF renderer runs.

## Quality Gate Summary

The following gates must pass before manuscript rendering is permitted. Results are captured by `scripts/z_generate_manuscript_varia` and injected here at render time.

| Gate | Result | Detail |
|---|---|---|
| ruff | 0 errors | Style and correctness linting across `src/` |
| ty | 0 diagnostics | Static type-checking via `ty check src/` |
| pytest | 625 passed | Full test suite (sec. Table 1), zero failures permitted |
| coverage | 87.0% | Branch coverage against 40% floor (matches sec. Table 1) |

All four gates must hold simultaneously. The `config.yaml testing.max_test_failures: 0` field enforces this constraint at the pipeline level — a single test failure blocks the render stage entirely, making a partial-pass manuscript certificate structurally impossible. The pytest count and coverage percentage reported in this table are the same values cited in the abstract and in sec. Table 1; they are emitted by a single pytest invocation whose JSON report feeds both sections, so no independent transcription is possible.

## Exact Reproduction Commands

To reproduce the colony_kernel test suite from a clean checkout:

```
# 1. Clone and enter the repository
git clone <repository-url> codomyrmex
cd codomyrmex

# 2. Install all dependencies (including dev extras)
uv sync

# 3. Run the full test suite - must yield 625 passing tests, 0 failures
HYPOTHESIS_NO_NPY=1 uv run pytest src/codomyrmex/tests/ -v \
    --cov=src/codomyrmex \
    --cov-report=term-missing \
    --cov-fail-under=40
```

```
# 4. Run only the colony_kernel sub-suite (faster isolation check)
uv run pytest src/codomyrmex/tests/unit/colony_kernel/ -v

# 5. Verify linting and type gates independently
uv run ruff check src/
uv run ty check src/

# 6. Regenerate manuscript variables from live pipeline output
uv run python scripts/z_generate_manuscript_variables.py \
    --config docs/manuscript/config.yaml \
    --out docs/manuscript_rendered/

# 7. Confirm all tokens resolved (non-empty output = gate failure)
grep -r "{{" docs/manuscript_rendered/ || echo "All tokens resolved"
```

The expected terminal summary after step 3 is `625 passed` with branch coverage at or above the 40% floor. Deviations indicate either a dependency mismatch (resolve with `uv sync` against the pinned `uv.lock`) or an environment difference captured in the software specification below.

## Software Environment Specification

Exact software versions are pinned in `uv.lock` at the repository root. The following table records the version range and precise version used to generate the certified results.

| Component | Version used | Minimum required | Notes |
|---|---|---|---|
| Python | 3.14.4 | 3.10 | CPython; tested on macOS arm64 and Ubuntu 22.04 x86_64 |
| uv | 0.4.0 | 0.4.0 | Dependency resolver and virtual-environment manager |
| pytest | 8.0 | 7.0 | Test runner; JSON report via `--json-report` |
| pytest-cov | 5.0 | 4.0 | Branch coverage measurement |
| ruff | 0.6.0 | 0.5.0 | Linting and formatting |
| ty | 0.1.0 | 0.1.0 | Static type checker |
| SQLite | system | 3.35 | In-process database for `TraceStore` and `ColonyMemory` |

Pinned dependency hashes are in `uv.lock`; reproduce the exact environment with:

```
uv sync --frozen   # Installs from lock file without re-solving
```

The `--frozen` flag guarantees that no dependency drift occurs between the environment that produced the certified test count and the environment used for verification.

## Simulation Specification

The colony simulation that produced the empirical results in sec. was run under the following initial conditions, recorded in `experiment_results.json` alongside all raw gate decisions, trust trajectories, pheromone snapshots, and budget traces.

| Parameter | Value | Source |
|---|---|---|
| Initial agent count | 5 | `config/colony_kernel/kernel.yaml` |
| Starting trust score (all agents) | 0.1 (sandbox floor) | `config/colony_kernel/roles.yaml` |
| Module registry size at launch | 8 MCP tool definitions | live filesystem |
| Scenario corpus — proposal count | 10 adversarial falsification vectors | `config/colony_kernel/falsification_` |
| Scenario corpus — source | Defined in sec. §5.6 | |
| Simulation tick count | Determined by scenario completion; see `experiment_results.json` | |
| Raw result artifact | `experiment_results.json` | Stage 1 output of pipeline (sec. §5.9) |

**Determinism guarantee.** The colony kernel contains no stochastic components: trust updates are closed-form Bayesian updates, pheromone decay is deterministic exponential decay applied once per tick, gate scores are deterministic functions of four measurable dimensions, and role promotions are threshold comparisons. No random number generator is seeded or invoked anywhere in `src/codomyrmex/colony_kernel/`. Consequently, identical inputs to `colony_analysis.py` will always produce bit-identical `experiment_results.json` output. Researchers wishing to verify this property can confirm it by inspecting `src/codomyrmex/colony_kernel/` for any call to `random`, `numpy.random`, or equivalent RNG APIs — none exist. The zero-mock, real-SQLite test suite further ensures that this determinism property holds under realistic I/O conditions, not only in isolation.

**What the provenance system does not guarantee.** The build-pipeline certificate — config hash, token injection, CI gate table, and generation timestamp — establishes that the manuscript was rendered from a specific, unmodified configuration and a clean codebase. It does *not* establish that the scenario corpus in §5.6 is representative of real-world agentic workloads, that the chosen initial trust score and agent count span the range of ecologically plausible colony sizes, or that the 10 adversarial vectors constitute a statistically complete coverage of the gate's failure modes. These are scientific-validity questions that lie outside the scope of a build-provenance certificate. Readers should treat the empirical results as characterising the system under the specific experimental conditions documented above, not as universally predictive of behaviour under arbitrary workloads.

## Zero-Mock Verification

The colony_kernel test suite operates under a strict zero-mock policy. All 625 passing tests use:

- **Real SQLite databases** — every `TraceStore` and `ColonyMemory` interaction persists to an in-process SQLite file created by pytest's `tmp_path` fixture. No in-memory mock replaces the database engine.
- **Real `TraceField` instances** — pheromone emission, decay, and signal routing are exercised on live `TraceField` objects, not stand-in dictionaries.
- **Real config loading** — `colony_kernel.config` is loaded from the actual YAML file on each test invocation. Profile state is never pre-seeded into a mock; it is read fresh from SQLite.

This policy is not merely a style preference — it is an epistemological requirement. The zero-mock constraint was what surfaced the **role-not-updating bug**: an agent's role was mutated in memory but the profile was loaded fresh from SQLite on each gate evaluation, silently discarding the mutation. A mock `ProfileStore` configured to return the updated role would have hidden this defect entirely. Real I/O forced the inconsistency into the open. Any future introduction of mocks must be treated as a reduction in the validity of the certificate this section asserts.

## Madlib Injection Verification

This manuscript demonstrates the template's token-injection ("madlib") capability: every quantitative claim is substituted from computed data at render time, not transcribed by hand.

**Which script**: `scripts/z_generate_manuscript_variables.py`

**Input files**:

- `docs/manuscript/config.yaml` — paper metadata, version, author list, keywords, experiment parameters
- `src/codomyrmex/colony_kernel/` — live filesystem scan for Python source file count and documentation file count
- pytest JSON report — test count and coverage percentage
- ruff and ty exit codes and JSON outputs

**Output directory**: `docs/manuscript_rendered/`

The script writes a single substitution map (key → value) and the rendering stage applies it to every `*.md` file in `docs/manuscript/`, writing resolved copies to `docs/manuscript_rendered/`. The substitution is applied with a single-pass regex replace; no token appears in the rendered output.

**The config.yaml token substitution pipeline as a reproducibility mechanism.** The madlib system is not merely a convenience — it is the primary mechanism by which this manuscript's numeric claims are reproducible. Every quantity cited in the abstract, results table, and gate summary traces to a named constant in `docs/manuscript/config.yaml` or to a computed value emitted by the pipeline. There is no disconnected prose claim that could silently diverge from the underlying measurement. Concretely: the gate weight formula (0.30 × budget + 0.30 × risk + 0.25 × trust + 0.15 × completeness) is defined once in `config.yaml` under `gates.weights` and referenced symbolically wherever it appears; a change to any weight coefficient is immediately reflected in all rendered sections or triggers an unresolved-token gate failure. Similarly, the test count (625) is not hand-typed — it is the `RESULT_TEST_COUNT` token emitted by the pytest JSON report at render time. A reader who disagrees with a reported number can trace it to its generating script in under two grep operations.

**To verify all tokens resolved**:

```
grep -r "{{" docs/manuscript_rendered/ || echo "All tokens resolved"
```

A non-empty match indicates an unresolved token and is treated as a gate failure. The generation timestamp embedded in this certificate is `2026-07-01T17:30:50Z`, confirming that the rendered copy was produced at a known point in time and not cached from a prior run.

# Scope, Related Work, and Positioning

## Agentic Software Engineering

The emergence of LLM-based software agents has produced a wave of frameworks for orchestrating tool-using models in engineering contexts. SWE-bench (Yang et al. 2024) established the first systematic evaluation of agent-generated patches against real GitHub issues, revealing that even strong models fail on the majority of tasks when operating without structured oversight. LangGraph (LangChain AI 2024) provides a graph-based state machine for multi-step agent pipelines, and AutoGen (Wu et al. 2023) enables conversational multi-agent workflows where agents negotiate task decomposition through natural language. CrewAI (CrewAI Inc. 2024) extends this model with role-based agent teams, pre-defined workflows, and hierarchical task delegation. All three frameworks excel at composing individual agent turns but lack a persistent, inspectable control plane: there is no first-class concept of trust accumulation, no cross-session consequence memory, and no signal-based coordination between concurrent agents.

Crucially, none of LangGraph, AutoGen, or CrewAI implement a pre-actuation falsification gate. In all three systems, an agent that has been granted a tool invocation right may exercise it on any proposal that passes syntactic validation — there is no mechanism for asking "has this agent's consequence history justified this class of action?" before execution proceeds. The ActuationGate addresses exactly this gap: every destructive proposal is evaluated against four measurable dimensions (budget, risk, trust, completeness) weighted 0.30 / 0.30 / 0.25 / 0.15, and only proposals that clear the composite threshold — calibrated against the agent's accumulated trust trajectory — proceed to execution. This falsification-before-actuation discipline is structurally absent from runtime-centric frameworks because those frameworks have no persistent consequence memory from which to compute a justified threshold.

Codomyrmex occupies precisely this gap. It is not a competing agent runtime — it is the missing control plane layer that sits above runtimes such as LangGraph and AutoGen. Where those frameworks answer "how do I route messages between agents?", Codomyrmex answers "which agents have earned the right to execute destructive actions, what have previous actuations cost, and where in the codebase is failure pheromone currently elevated?" The ActuationGate, TraceField, and consequence memory together constitute a governance substrate that any MCP-compatible runtime can query and respect.

## What Codomyrmex Is Not

Explicit scope statements prevent mischaracterization of the contribution. The following are deliberate exclusions, not implementation gaps.

**Not a general-purpose agent framework.** Codomyrmex does not implement agent execution, message routing, tool dispatch, or task decomposition. It is a governance and coordination layer. Researchers seeking a full agent runtime should use LangGraph (LangChain AI 2024), AutoGen (Wu et al. 2023), or CrewAI (CrewAI Inc. 2024) and optionally layer Codomyrmex above them for trust-gated actuation control.

**Not a replacement for LLM APIs.** Codomyrmex does not wrap or abstract LLM inference. It has no opinion on which model generates proposals — only on whether those proposals, once generated, meet the evidentiary standard required for destructive actuation. Model selection, prompt engineering, and inference costs remain entirely outside the system boundary.

**Not evaluated on production workloads.** The empirical results in sec.  are derived from a controlled experimental setup with 5 agents, 10 adversarial falsification vectors, and a fixed scenario corpus. No claims are made about behavior under real-world traffic patterns, adversarial users with persistent access, or multi-tenant deployments. The actuation gate has been validated to be correct and safe under these controlled conditions; extrapolation to production workloads requires independent evaluation and is an identified future direction.

**Not a security boundary.** The ActuationGate is a governance mechanism grounded in consequence history, not a cryptographic access-control system. A sufficiently persistent agent that accumulates enough positive interactions can eventually earn destructive

access. The system is designed to make this process transparent and auditable, not to prevent it. Readers seeking cryptographic capability confinement should treat the ActuationGate as complementary to, not a replacement for, OS-level sandboxing, network egress controls, and capability-based security substrates such as those discussed in sec. .

**Not a distributed system.** The current implementation uses a single-process SQLite backend. Multi-repository colony state, shared pheromone fields across machines, and distributed consensus for trust scores are all out of scope for this paper.

## Stigmergy and Self-Organizing Systems

Grassé's original term (Grassé 1959) described how ants coordinate without central control — work done by one individual modifies the environment in a way that stimulates further work. Parunak's foundational work on digital pheromones (Parunak 1997) showed that synthetic stigmergy could govern distributed software agents without central coordination, and Dorigo & Stützle's comprehensive treatment of Ant Colony Optimization (Dorigo and Stützle 2004b) established the computational theory underlying pheromone-guided search: decay rates, evaporation schedules, and reinforcement dynamics that balance exploitation of known-good paths against exploration of alternatives. These are the direct computational ancestors of the TraceField and ColonySignal design.

The Colony Kernel implements digital stigmergy via TraceField pheromone deposits keyed by (location, signal_type). Unlike biological systems where signals are continuous diffusion fields, the Colony Kernel uses discrete strength values with configurable exponential decay — more predictable, less emergent, but verifiable. The decay schedule borrows directly from Dorigo's evaporation parameter: deposits weaken each tick, preventing stale success signals from permanently masking emergent failure patterns.

## Multi-Agent Systems and Trust

Classical MAS literature (Wooldridge & Jennings (Wooldridge and Jennings 1995)) distinguished between architectures but seldom addressed how trust accumulates over time in an operative sense. The foundational conceptual work comes from Marsh's formalization of computational trust (Marsh 1994), which decomposed trust into competence, benevolence, and integrity dimensions — a tripartite structure that maps directly onto the Colony Kernel's consequence memory: competence is approximated by the action success rate, integrity by the human_feedback multiplier, and benevolence by the pattern of proactive vs. reactive failure responses.

For agents with unknown histories — which is the common case for newly provisioned LLM agents — bootstrapping trust is a distinct problem from updating trust given evidence. The FIRE model (Huynh et al. 2006) addresses this through interaction, witness, role-based, and certified trust components, providing mechanisms to derive initial trust estimates from indirect evidence when direct interaction history is sparse. Burnett et al. (Burnett et al. 2013) extend this to agents operating across heterogeneous environments, showing that referral networks can seed trust scores before any direct observation. In the Colony Kernel, this bootstrapping problem is currently handled conservatively: new agents begin below the actuation threshold and must accumulate direct consequence history before the gate opens. Integrating FIRE-style indirect evidence propagation is an identified future extension.

Sabater & Sierra's survey (Sabater and Sierra 2005) of trust and reputation systems provides the conceptual grounding for trust_score computation from consequence history, situating the Colony Kernel's approach within the broader trajectory from purely social reputation systems toward hybrid models that combine direct experience with structural context.

## Model Context Protocol

The MCP specification (Anthropic, 2024 (Anthropic 2024)) defines a JSON-RPC protocol for tool exposure. Codomyrmex exposes all Colony Kernel subsystems as MCP tools, meaning the gate can be queried and the pheromone field inspected from any MCP-compatible LLM client. This integration is in-scope; MCP server deployment and transport configuration are out of scope.

## Reinforcement Learning, Constitutional AI, and Consequence Memory

Sutton & Barto (Sutton and Barto 2018) formalize the reward-feedback loop that consequence memory approximates. The Colony Kernel does not implement policy gradients — trust_delta is a hand-crafted heuristic (success → +0.04, failure → -0.08, human_feedback multiplier) rather than a learned policy. Connecting consequence memory to a proper RL policy optimizer is explicitly deferred.

Two more recent lines of work sharpen the relationship between consequence memory and learned alignment mechanisms. Constitutional AI (CAI) (Bai et al. 2022) and RLHF (Christiano et al. 2017) train models to internalize normative constraints through a reward signal derived from human preference feedback. The Colony Kernel's trust score dynamics are structurally analogous: the human_feedback multiplier in the trust update formula plays the role of a preference signal, and the accumulation of consequence records over time resembles the dataset that reward modeling optimizes against. The key difference is that CAI/RLHF modify the underlying model weights — their alignment signal is baked into the parameters. The Colony Kernel leaves model weights unchanged and instead maintains an external, auditable consequence ledger that gates actuation at inference time. This makes the governance layer model-agnostic and revocable: a trust score can be reset, audited, or overridden by a human operator without touching the model. Whether external consequence memory or internalized reward modeling produces more robust alignment under adversarial conditions is an open empirical question.

**The actuation gate as a process reward model.** Process reward models (PRMs) (Lightman et al. 2023; Uesato et al. 2022) assign scalar scores to intermediate reasoning steps rather than only to final outputs, enabling fine-grained feedback during multi-step problem solving. The ActuationGate is functionally a process reward model applied to agent action proposals: it scores proposals

along four dimensions (budget, risk, trust, completeness) weighted 0.30 / 0.30 / 0.25 / 0.15, producing a composite score that determines whether execution proceeds. Unlike learned PRMs trained on human annotations, the ActuationGate's scoring function is hand-crafted and deterministic. But the architectural pattern is the same — a step-level evaluator interposed between proposal generation and execution. This connection suggests a concrete upgrade path: replacing the hand-crafted scoring function with a learned PRM trained on accumulated consequence logs, inheriting the PRM literature's training and calibration methodology while retaining the Colony Kernel's external, auditable governance structure. This integration is an identified future direction.

## Capability-Based Security and Least Authority

The ActuationGate instantiates a well-established security architecture that deserves explicit acknowledgment. Miller's capability-based security model (Miller and Shapiro 2003) argues that access rights should be unforgeable tokens carried by callers, not checked against ambient authority tables — a shift from identity-based ("who are you?") to capability-based ("what do you hold?") authorization. Saltzer and Schroeder's Principle of Least Authority (POLA) (Saltzer and Schroeder 1975) states that every component should operate with the minimum set of privileges needed to accomplish its task, and authority should be granted incrementally as need is demonstrated.

The ActuationGate enforces both principles. Trust tokens are earned through consequence history rather than conferred by identity: an agent's trust_score is an unforgeable capability accumulated from verifiable past interactions, not a static credential that can be spoofed. Destructive tools require a trust_score above a configurable threshold — authority is granted incrementally as the agent's consequence record justifies it. This is POLA applied to the agentic context: LLM agents begin with read-only access and graduate toward write and execute authority only as their consequence memory warrants. Security venues reviewing this work should recognize the ActuationGate as a concrete instantiation of least-authority principles, extended to cover the distinct challenge of agents whose competence distribution is unknown at provisioning time.

## Active Inference and Free Energy

Friston's Free Energy Principle (Friston 2010) frames intelligent behavior as minimization of variational free energy — the gap between an agent's generative model of the world and its sensory observations. A full mapping of the Colony Kernel onto this framework requires identifying three components: (1) a generative model $p(o, s)$ over observations $o$ and hidden states $s$; (2) a variational posterior $q(s)$ that the agent maintains over those hidden states; and (3) a policy-selection mechanism that minimizes expected free energy over future states.

A rigorous active inference treatment of the Colony Kernel is beyond the scope of this paper and is deferred to future work. What can be noted structurally is that the pressure loop shares the phenomenology of free energy minimization — high failure pheromone concentrations correspond to regions of elevated prediction error, and agent routing toward those regions resembles the exploratory behavior that active inference predicts when epistemic value is high. We do not claim formal equivalence. Readers who find the analogy productive should treat it as a design intuition rather than a theoretical commitment; readers seeking formal guarantees should await the companion mathematical treatment.

## Explicit Scope Limitations

The following limitations are deliberate design decisions, not oversights. Each reflects a trade-off made to keep the initial system tractable and verifiable; each names a concrete future extension.

**Control plane only.** The Colony Kernel governs agent behavior but does not execute actions itself. This boundary is load-bearing: mixing governance and execution in a single component makes both harder to audit. LLM runtimes (LangGraph, AutoGen, or direct MCP clients) sit below the Colony Kernel and handle execution; the Colony Kernel sits above them and handles authority. A future integration layer could make this separation transparent to callers.

**Discrete-tick time model.** Pheromone decay advances on `ColonyKernel.tick()` calls rather than on a wall-clock timer. This was chosen deliberately: a tick-driven model is deterministic, testable, and reproducible across environments without scheduling infrastructure. Real-time decay introduces non-determinism that complicates replay and debugging. Integration with a real-time clock daemon — mapping ticks to configurable wall-clock intervals — is a straightforward future extension once the tick semantics are proven stable.

**Heuristic trust deltas, no learned policy.** Trust scoring uses fixed deltas (success → +0.04, failure → -0.08, human_feedback multiplier) rather than a policy learned from data. This choice follows the principle Sutton & Barto (Sutton and Barto 2018) articulate for bootstrapping reinforcement learning systems: hand-crafted reward shaping provides stable initial signal while learned policies are still sample-starved. The heuristic deltas were calibrated to match the intuition that failures should cost more than successes gain — a risk-averse bias appropriate for irreversible actions. Replacing the heuristic with a proper policy gradient or Q-learning update is an identified future direction, contingent on accumulating sufficient consequence logs to train from.

**Human-confirmed pruning.** The pruning daemon identifies stale consequence records and low-activity code regions but never archives automatically. Automated pruning of operational memory raises correctness risks that are difficult to bound without extensive deployment experience. Human confirmation is required until a sufficient empirical record justifies a configurable auto-prune threshold.

**Synchronous MCP tools.** The current MCP surface is synchronous, which means concurrent proposals from multiple agents serialize at the gate. This is conservative and safe; it does not reflect a fundamental design constraint. Async concurrency across simultaneous proposals — with per-proposal trust evaluation and pheromone read isolation — is the primary scalability target for a future release.

**Single-process consequence memory.** The SQLite backend stores consequence history in a single-process database, making distributed colony state across multiple repositories or machines out of scope. Distributed consensus for pheromone fields and shared trust scores is a substantial engineering problem; the current design optimizes for correctness and debuggability in the single-repository case. Sharding or replication strategies will be addressed in a follow-on architecture document.

# References

The bibliography for this manuscript is maintained in `docs/manuscript/references.bib` using BibTeX format. Pandoc processes it during compilation via the `--natbib` flag, which delegates citation rendering to the `natbib` LaTeX package. All entries in `references.bib` must be valid BibTeX; validate before submission using:

```
uv run python -m infrastructure.reference.citation.cli validate \
    docs/manuscript/references.bib --strict
```

Citation syntax throughout the manuscript uses the Pandoc-native `[@key]` form (e.g. `[@smith2024]` for a single reference, `[@smith2024; @jones2023]` for multiple). Raw `\cite{}` commands must not appear in Markdown source — they bypass Pandoc's citation pipeline and break non-LaTeX output targets.

## Bibliography Coverage

The bibliography contains 28 entries spanning the five primary citation domains of this manuscript: stigmergy and self-organizing systems, multi-agent trust and reputation, reinforcement learning and consequence memory, capability-based security and least authority, and agentic software engineering frameworks.

**Citation key inventory** (all keys cited in the manuscript body have corresponding entries in `references.bib`):

| Key | Entry | Domain |
|---|---|---|
| `friston2010free` | Friston 2010 — Free Energy Principle | Active Inference |
| `grasse1959reconstruction` | Grassé 1959 — Stigmergy original formulation | Stigmergy |
| `parunak1997pheromones` | Parunak 1997 — Digital pheromones for distributed software agents | Stigmergy |
| `dorigo2004aco` | Dorigo & Stützle 2004 — Ant Colony Optimization | Stigmergy / ACO |
| `bonabeau1999swarm` | Bonabeau et al. 1999 — Swarm Intelligence | Stigmergy |
| `wooldridge1995intelligent` | Wooldridge & Jennings 1995 — Intelligent Agents | MAS |
| `marsh1994trust` | Marsh 1994 — Formalising Trust as a Computational Concept | MAS Trust |
| `huynh2006fire` | Huynh et al. 2006 — FIRE integrated trust and reputation model | MAS Trust |
| `burnett2013bootstrapping` | Burnett et al. 2013 — Bootstrapping trust via stereotypes | MAS Trust |
| `kamvar2003eigentrust` | Kamvar et al. 2003 — EigenTrust reputation management | MAS Trust |
| `sabater2005review` | Sabater & Sierra 2005 — Review of computational trust models | MAS Trust |
| `sutton2018reinforcement` | Sutton & Barto 2018 — Reinforcement Learning: An Introduction | RL |
| `miller2003capabilities` | Miller & Shapiro 2003 — Capability-based security | Security |
| `saltzer1975protection` | Saltzer & Schroeder 1975 — Principle of Least Authority (POLA) | Security |
| `yang2024swebench` | Yang et al. 2024 — SWE-bench: LLM patch generation benchmark | Agentic SE |
| `langgraph2024` | LangChain AI 2024 — LangGraph state machine for agentic pipelines | Agentic SE |
| `wu2023autogen` | Wu et al. 2023 — AutoGen conversational multi-agent framework | Agentic SE |

| Key | Entry | Domain |
|---|---|---|
| `anthropic2024mcp` | Anthropic 2024 — Model Context Protocol specification | MCP |
| `kauffman1993origins` | Kauffman 1993 — Origins of Order | Self-Organization |
| `apt2003principles` | Apt 2003 — Principles of Constraint Programming | Constraint Programming |
| `popper1959logic` | Popper 1959 — Logic of Scientific Discovery | Falsification |
| `peng2011reproducible` | Peng 2011 — Reproducible Research in Computational Science | Reproducibility |
| `dorigo2004ant` | Dorigo & Stützle 2004 — Ant Colony Optimization (alternate key) | Stigmergy / ACO |
| `crewai2024` | CrewAI 2024 — Role-based agent teams and hierarchical task delegation | Agentic SE |
| `bai2022constitutional` | Bai et al. 2022 — Constitutional AI: Harmlessness from AI Feedback | RL / Alignment |
| `christiano2017deep` | Christiano et al. 2017 — Deep Reinforcement Learning from Human Preferences | RL / Alignment |
| `lightman2023let` | Lightman et al. 2023 — Let's Verify Step by Step (Process Reward Models) | RL / PRMs |
| `uesato2022solving` | Uesato et al. 2022 — Solving Math Word Problems with Process- and Outcome-Based Feedback | RL / PRMs |

**Unresolved keys:** none. Every `[@key]` citation in the manuscript body has a corresponding entry in `references.bib`. Run the validation command above before submission to confirm.

**Coverage note:** The 12 trust and stigmergy references represent the theoretical core. The agentic SE trio (SWE-bench, LangGraph, AutoGen) anchors the positioning claim that existing frameworks lack persistent consequence memory — a claim that is empirically grounded by the absence of trust-accumulation primitives in those systems' documented APIs. The Saltzer & Schroeder and Miller capability-security citations connect the ActuationGate design to a 50-year lineage of least-authority engineering, establishing that the gate is not a novel ad hoc mechanism but a concrete instantiation of well-understood security principles applied to a new threat surface.


Anthropic. 2024. *Model Context Protocol: An Open Standard for Connecting AI Assistants to the Systems Where Data Lives*. Anthropic. https://modelcontextprotocol.io.

Apt, Krzysztof R. 2003. *Principles of Constraint Programming*. Cambridge University Press.

Bai, Yuntao, Andy Jones, Kamal Ndousse, et al. 2022. "Constitutional AI: Harmlessness from AI Feedback." *arXiv Preprint arXiv:2212.08073*. https://arxiv.org/abs/2212.08073.

Bonabeau, Eric, Marco Dorigo, and Guy Theraulaz. 1999. *Swarm Intelligence: From Natural to Artificial Systems*. Oxford University Press.

Burnett, Colin, Timothy J. Norman, and Katia Sycara. 2013. "Bootstrapping Trust Evaluations Through Stereotypes." *Proceedings of the 12th International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)* (St. Paul, MN), 437–44.

Christiano, Paul F., Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. "Deep Reinforcement Learning from Human Preferences." *Advances in Neural Information Processing Systems (NeurIPS)* 30. https://arxiv.org/abs/1706.03741.

CrewAI Inc. 2024. *CrewAI: Framework for Orchestrating Role-Playing, Autonomous AI Agents*. Https://github.com/crewAIInc/crewAI.

Dorigo, Marco, and Thomas Stützle. 2004a. *Ant Colony Optimization*. MIT Press.

Dorigo, Marco, and Thomas Stützle. 2004b. *Ant Colony Optimization*. MIT Press.

Friston, Karl. 2010. "The Free-Energy Principle: A Unified Brain Theory?" *Nature Reviews Neuroscience* 11 (2): 127–38. https://doi.org/10.1038/nrn2787.

Grassé, Pierre-Paul. 1959. "La Reconstruction Du Nid Et Les Coordinations Inter-Individuelles Chez Bellicositermes natalensis Et Cubitermes Sp. La Théorie de La Stigmergie: Essai d'interprétation Du Comportement Des Termites Constructeurs." *Insectes Sociaux* 6 (1): 41–80. https://doi.org/10.1007/BF02223791.

Huynh, Trung Dong, Nicholas R. Jennings, and Nigel Shadbolt. 2006. "An Integrated Trust and Reputation Model for Open Multi-Agent Systems." *Autonomous Agents and Multi-Agent Systems* 13 (2): 119–54. https://doi.org/10.1007/s10458-005-6825-4.

LangChain AI. 2024. *LangGraph: Build Stateful, Multi-Actor Applications with LLMs*. Https://github.com/langchain-ai/langgraph.

Lightman, Hunter, Vineet Kosaraju, Yura Burda, et al. 2023. "Let's Verify Step by Step." *arXiv Preprint arXiv:2305.20050*. https://arxiv.org/abs/2305.20050.

Marsh, Stephen Paul. 1994. "Formalising Trust as a Computational Concept." PhD thesis, University of Stirling.

Miller, Mark S., and Jonathan S. Shapiro. 2003. *Paradigm Regained: Abstraction Mechanisms for Access Control*. Johns Hopkins University.

Parunak, H. Van Dyke. 1997. ""Go to the Ant": Engineering Principles from Natural Multi-Agent Systems." *Annals of Operations Research* 75: 69–101. https://doi.org/10.1023/A:1018980001403.

Sabater, Jordi, and Carles Sierra. 2005. "Review on Computational Trust and Reputation Models." *Artificial Intelligence Review* 24 (1): 33–60. https://doi.org/10.1007/s10462-004-0595-7.

Saltzer, Jerome H., and Michael D. Schroeder. 1975. "The Protection of Information in Computer Systems." *Proceedings of the IEEE* 63 (9): 1278–308. https://doi.org/10.1109/PROC.1975.9939.

Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press.

Uesato, Jonathan, Nate Kushman, Ramana Kumar, et al. 2022. "Solving Math Word Problems with Process- and Outcome-Based Feedback." *arXiv Preprint arXiv:2211.14275*. https://arxiv.org/abs/2211.14275.

Wooldridge, Michael, and Nicholas R. Jennings. 1995. "Intelligent Agents: Theory and Practice." *The Knowledge Engineering Review* 10 (2): 115–52. https://doi.org/10.1017/S0269888900008122.

Wu, Qingyun, Gagan Bansal, Jieyu Zhang, et al. 2023. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." *arXiv Preprint arXiv:2308.08155*. https://arxiv.org/abs/2308.08155.

Yang, John, Carlos E. Jimenez, Alexander Wettig, et al. 2024. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" *arXiv Preprint arXiv:2310.06770*. https://arxiv.org/abs/2310.06770.